PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

Smart Inventory & Supply Chain Tracker

Submitted by:

PES1UG24AM217	Rahul Prasanna	5D
PES1UG24AM206	Prithvish Adhikari	5D

Class of Prof. ***Devna Sumesh***
[Lab Faculty]

Abstract

Small and mid-sized distributors often keep stock in several warehouses and buy from many suppliers, yet track it in spreadsheets and messages. Stock totals drift when a transfer or delivery is only half-recorded, low stock is noticed too late, and supplier quality and statutory certificates (FSSAI, BIS, GST) go unmonitored.

This project builds a **Smart Inventory & Supply Chain Tracker**: a web application that keeps stock consistent across multiple warehouses, manages the full purchase-order lifecycle, raises low-stock alerts automatically, and scores supplier quality. Structured, transactional data (products, warehouses, inventory, purchase orders, stock movements, users) is stored in **PostgreSQL 18**, where **constraints, triggers, stored procedures with row-level locking, views and full-text search** enforce the business rules inside the database. Data whose shape varies from record to record (category-specific product specifications, supplier certificates and delivery feedback) is stored in **MongoDB Atlas**, validated with `$jsonSchema` and analysed with **aggregation pipelines**. A **Next.js 16** application provides forms, tables, search, filters and dashboards through an API layer, with bcrypt-hashed passwords, JWT sessions and three roles (Admin, Manager, Supplier).

The result keeps stock correct even under concurrent use: every change is atomic and logged, and the movement log always matches the inventory. Each database is used for the kind of data it handles best, and the two are kept consistent with each other.

Keywords: PostgreSQL, MongoDB, Next.js, transactions, triggers, stored procedures, full-text search, aggregation pipeline, inventory management.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement b) Novelty c) Team and Responsibilities	4
2.	User Requirements Specification: Tech Stack a) Technology stack and architecture b) Why SQL and why NoSQL c) Security	6
3.	E-R Model a) List of Entities [no weak entities] b) List of non-simple Attributes c) ER Diagram Snapshot d) Relationships and Cardinalities	9
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram c) Mapping and Normalisation	12
5.	SQL DDL Statements	15
6.	SQL DML Statements	18
7.	NoSQL Database Commands (MongoDB)	21
8.	Results a) Resulting Tables' Screenshots b) Validation and Error Handling c) Test Cases and Results	25
9.	Next.js: Project Tree Structure	31
10.	Few Sample Screenshots of Application Code and Demo	33
11.	Evaluation Rubrics: Completion Status	41
A.	Annexure A: Stock Report	44

1. Introduction

1.1 Problem Statement

Project title	**Smart Inventory & Supply Chain Tracker**
Category	DBMS Mini Project · Option 1: SQL + NoSQL
GitHub repository	**https://github.com/RahulPrasanna22/DBMS-Mini_Project**

Short description. A multi-warehouse inventory and procurement system with supplier-quality analytics, built on PostgreSQL (SQL) and MongoDB (NoSQL) with a Next.js web interface.

Small and mid-sized distributors often keep stock in several warehouses and buy from many suppliers, yet track it in spreadsheets and messages. Stock totals drift when a transfer or a delivery is only half-recorded, low stock is noticed too late, and supplier quality and statutory certificates (FSSAI, BIS, GST) go unmonitored. The problem addressed here is to keep stock **correct and auditable** across warehouses, automate re-ordering signals, and measure supplier quality from real deliveries, using the right database for each kind of data.

The design matches how Indian distributors work: several city warehouses with limited space, food and electronics suppliers who need statutory certificates (FSSAI, BIS, GST), and supplier performance tracked from actual deliveries rather than from memory.

1.2 Novelty

> **In one line:** most inventory projects are CRUD screens over a few tables, with the rules checked in the UI. In this project the **databases enforce the rules themselves**, stock stays correct even under concurrent use, and two databases are used for what each does best, **kept consistent with each other**, to measure supplier quality as well as stock.

#	Typical inventory project	This project
1	Rules (no negative stock, capacity) checked in the UI or not at all	**Database-enforced rules.** CHECK constraints and a BEFORE trigger enforce capacity across *all* products in a warehouse. No code path, not even a direct SQL statement, can break them. The database's own error message is shown in the UI, so the validation is visible.
2	Low stock spotted by someone looking at a list	**Self-managing alerts.** An AFTER trigger raises an alert when stock reaches the reorder level and *resolves it automatically* when stock recovers, inside the same transaction as the stock change. A partial index keeps the open-alert lookup fast.
3	Stock updated with two separate statements; concurrent users can corrupt totals	**Deadlock-free concurrent transfers.** `transfer_stock` locks both inventory rows with `FOR UPDATE` in a **fixed order** (by warehouse id). Two opposite transfers at the same moment can neither deadlock nor oversell, and any failure rolls back everything.
4	A second database added "for the requirement", or a schema-less dump	**Each database is chosen for its data, and the flexible data is still validated.** Product specs differ per category, yet MongoDB's `$jsonSchema` with `oneOf` still requires the right fields for each category (weight and shelf life for Grocery; warranty for Electronics).

#	Typical inventory project	This project
5	Two databases that do not know about each other	**Cross-database integrity.** Feedback is accepted only for a purchase order that exists in PostgreSQL, belongs to that supplier and is RECEIVED, so there are no fake or duplicate reviews. The MongoDB average is written back to `suppliers.rating`, and a new document's ObjectId is written back to `mongo_doc_id`: a two-way link.
6	Suppliers are only a name and phone number	**Supplier quality and compliance analytics.** Aggregation pipelines produce a scorecard (worst first), the most frequent delivery problems, and certificates (FSSAI, BIS, GST) expiring within N days, computed with `$dateDiff`.
7	One "admin" login, or roles checked only in the menu	**Supplier self-service with data isolation.** Suppliers log in and see only their own products, orders and documents. Every query filters by their `supplier_id`, and another supplier's data returns "not found", so IDs cannot be probed.
8	Stock is a single number that is overwritten	**Auditable stock ledger.** Every change is logged in `stock_movements`. The full stock history can be rebuilt from this log, and it always matches the inventory because the procedures update both in the same transaction.
9	The whole app fails if one service is down	**Graceful degradation.** If MongoDB Atlas cannot be reached, product and supplier pages still show their PostgreSQL data with a warning instead of an error page.

1.3 Team and Responsibilities

Name	SRN	Primary responsibility
Rahul Prasanna	PES1UG24AM217	Next.js application (auth, roles, products, inventory, purchase orders, dashboard, admin), PostgreSQL integration, testing
Prithvish Adhikari	PES1UG24AM206	Database design (schema, triggers, procedures, views), MongoDB (Atlas, validators, pipelines), supplier-quality module, documentation

2. User Requirements Specification: Tech Stack

2.1 Technology stack

Layer	Technology	Purpose in this project
Frontend	Next.js 16.4 (App Router), React 19, Tailwind CSS 4, Recharts	Pages, forms, tables, search, filters, dashboard charts
Backend / API	Next.js Route Handlers (Node.js 20+)	REST endpoints in `app/api/**/route.js`; all database access goes through them
SQL database	PostgreSQL 18 + `pg` 8 (node-postgres)	Products, warehouses, inventory, purchase orders, movements, users; triggers, procedures, views; full-text search
NoSQL database	MongoDB Atlas (M0, Mumbai) + `mongodb` 7 driver	Product specifications, supplier certificates and delivery feedback; validators and aggregation pipelines
Authentication	bcryptjs 3, jose 6 (JWT)	Password hashing; signed session cookie (httpOnly, 8 h); 3 roles
Configuration	`.env.local` (git-ignored)	DATABASE_URL, MONGODB_URI, MONGODB_DB, JWT_SECRET; nothing hard-coded
Version control	Git + GitHub	Source code, individual commits, README
Tools	psql, pgAdmin 4, MongoDB Atlas UI, VS Code	Running scripts, inspecting data

2.2 System architecture

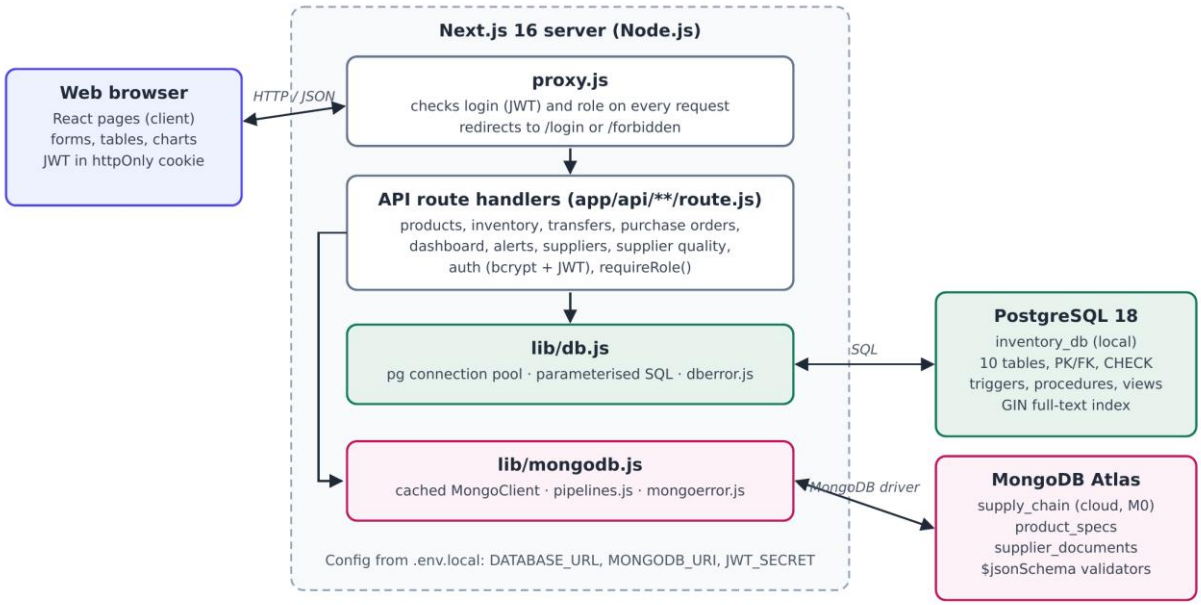


Figure 2.1: System architecture

Layer	Responsibility
Presentation	Next.js App Router pages (React client components) with Tailwind CSS and Recharts. Pages call the API with `fetch` and show database errors in a red alert.
Access control	`proxy.js` runs before every request: verifies the JWT cookie, redirects unauthenticated users to `/login`, and sends users without the right role to `/forbidden` (or returns 401 for API calls).
API	Route handlers in `app/api/**/route.js`. Each one calls `requireRole(...)`, validates its input, runs parameterised SQL or MongoDB operations, and returns JSON.
Data access	`lib/db.js` (shared `pg` pool), `lib/mongodb.js` (cached client), `lib/pipelines.js`, and error mapping that turns database errors into HTTP 400 responses with readable messages.
Data	PostgreSQL 18 (local) and MongoDB Atlas (cloud). Connection strings come from environment variables.

2.3 Why SQL and why NoSQL

The project uses **Option 1: SQL + NoSQL**. The rule followed was simple: if the data has a fixed structure, takes part in transactions or needs referential integrity, it goes to PostgreSQL. If its structure varies from record to record and it is read mostly as a whole document, it goes to MongoDB.

Data	Database	Reason
Users, roles	PostgreSQL	Fixed structure; the role CHECK constraint and the supplier foreign key must always hold.
Products, categories, suppliers, warehouses	PostgreSQL	Master data referenced by many tables; uniqueness (SKU, email) and foreign keys are essential.
Inventory, stock movements	PostgreSQL	Numeric quantities that must never go negative or exceed capacity; changed by multi-row transactions with row locks.
Purchase orders and items	PostgreSQL	Header and line items must be written together (ACID); status and date rules need CHECK constraints.
Low-stock alerts	PostgreSQL	Generated by triggers inside the same transaction as the stock change, so they can never be out of sync.
Product specifications	MongoDB	Attributes differ completely by category (rice: weight, grain, shelf life, nutrition; cable: connector, length, wattage). In SQL this would need many nullable columns or an EAV table. In MongoDB each product stores exactly its own fields, and a validator still enforces the required fields per category.
Supplier certificates, contacts	MongoDB	Certificate types vary by supplier (FSSAI for food, BIS for electronics, GST, ISO). Each certificate list is read and shown as a whole.
Delivery feedback	MongoDB	Free-text comments with a variable list of issue tags, growing over time as a nested array; analysed with aggregation pipelines (`$unwind`, `$group`).

How the two databases are linked

Each MongoDB document stores the PostgreSQL key it belongs to (`product_id` or `supplier_id`, both uniquely indexed). In the other direction, PostgreSQL stores the document's ObjectId in

`products.mongo_spec_id` and `suppliers.mongo_doc_id`. MongoDB has no foreign keys, so cross-database rules are checked in the API layer. For example, feedback is only accepted for a purchase order that exists in PostgreSQL, belongs to that supplier and has status `RECEIVED`.

Why not vector search?
The brief lists vector search as optional. This project chose Option 1 because its unstructured data (specifications, certificates, feedback) is queried by exact fields and aggregated by tags and dates, not searched by meaning. Text search over products is handled by PostgreSQL full-text search. Section 11.4 lists semantic search over feedback as a future enhancement.

2.4 Security

- **Passwords:** stored only as bcrypt hashes (cost factor 10) and compared with `bcrypt.compare`.
- **Sessions:** a signed JWT (HS256, 8-hour expiry) in an `httpOnly`, `sameSite=lax` cookie that JavaScript cannot read; `secure` in production.
- **SQL injection:** every query uses `$1, $2 …` placeholders. The only dynamic SQL is `ORDER BY`, which is limited to a whitelist of column names.
- **Credentials:** stored in `.env` / `.env.local`, which `.gitignore` excludes. The repository only contains `.env.example` with placeholders. After a real password was accidentally committed during development, the Atlas password was changed and the file was cleaned.
- **Authorisation:** checked both in `proxy.js` (pages) and in every API route (`requireRole`). Suppliers are additionally filtered to their own `supplier_id` in every query, and they get the same "not found" response for another supplier's data, so they cannot probe IDs.

3. E-R Model

3.1 List of Entities

The relational part of the system has eight entities. **No weak entities** are needed: every entity has its own surrogate primary key (`SERIAL` / `BIGSERIAL`). `STOCK_MOVEMENT` and `LOW_STOCK_ALERT` refer to a product and a warehouse, but they are identified by their own key, so they are strong entities that take part in the ternary relationships *records* and *raises*.

Entity	Primary key	Attributes	Type
USER	`user_id`	username, password_hash, role, supplier_id, created_at	Strong
SUPPLIER	`supplier_id`	name, email, phone, city, rating, active, mongo_doc_id	Strong
CATEGORY	`category_id`	name	Strong
PRODUCT	`product_id`	sku, name, description, unit_price, reorder_level, mongo_spec_id, search	Strong
WAREHOUSE	`warehouse_id`	name, city, capacity	Strong
PURCHASE_ORDER	`po_id`	status, order_date, expected_date, received_date	Strong
STOCK_MOVEMENT	`movement_id`	change_qty, movement_type, reference_id, created_at	Strong
LOW_STOCK_ALERT	`alert_id`	quantity, resolved, created_at	Strong

3.2 Non-simple Attributes

Most attributes are simple and atomic. The non-simple ones are listed below, together with where they live in the design.

Attribute	Entity	Kind	How it is handled
Order total value	PURCHASE_ORDER	Derived	Not stored. Computed as `SUM(quantity × unit_cost)` over the order's items.
Stock value	PRODUCT	Derived	Not stored. Computed in the view `v_stock_summary` as total stock × unit price.
Warehouse utilisation	WAREHOUSE	Derived	Not stored. Units stored ÷ capacity, computed in a query with a LEFT JOIN.
search	PRODUCT	Derived (generated)	A `tsvector` generated from name and description, so it can never go out of date; GIN-indexed.
rating	SUPPLIER	Derived (stored)	Average of MongoDB feedback ratings, written back to PostgreSQL when feedback is recorded.
specs, images	PRODUCT	Multivalued / composite	Category-specific and variable. Stored as one document in MongoDB `product_specs`, not in PostgreSQL (1NF).

Attribute	Entity	Kind	How it is handled
nutrition_per_100g	PRODUCT (Grocery)	Composite	Nested sub-document (energy_kcal, protein_g, carbs_g) inside `specs` in MongoDB.
certifications, contacts	SUPPLIER	Multivalued	Arrays inside the supplier's MongoDB document in `supplier_documents`.
feedback (issues)	SUPPLIER	Multivalued, nested	Array of feedback items, each with its own array of issue tags, in `supplier_documents`.

3.3 ER Diagram Snapshot

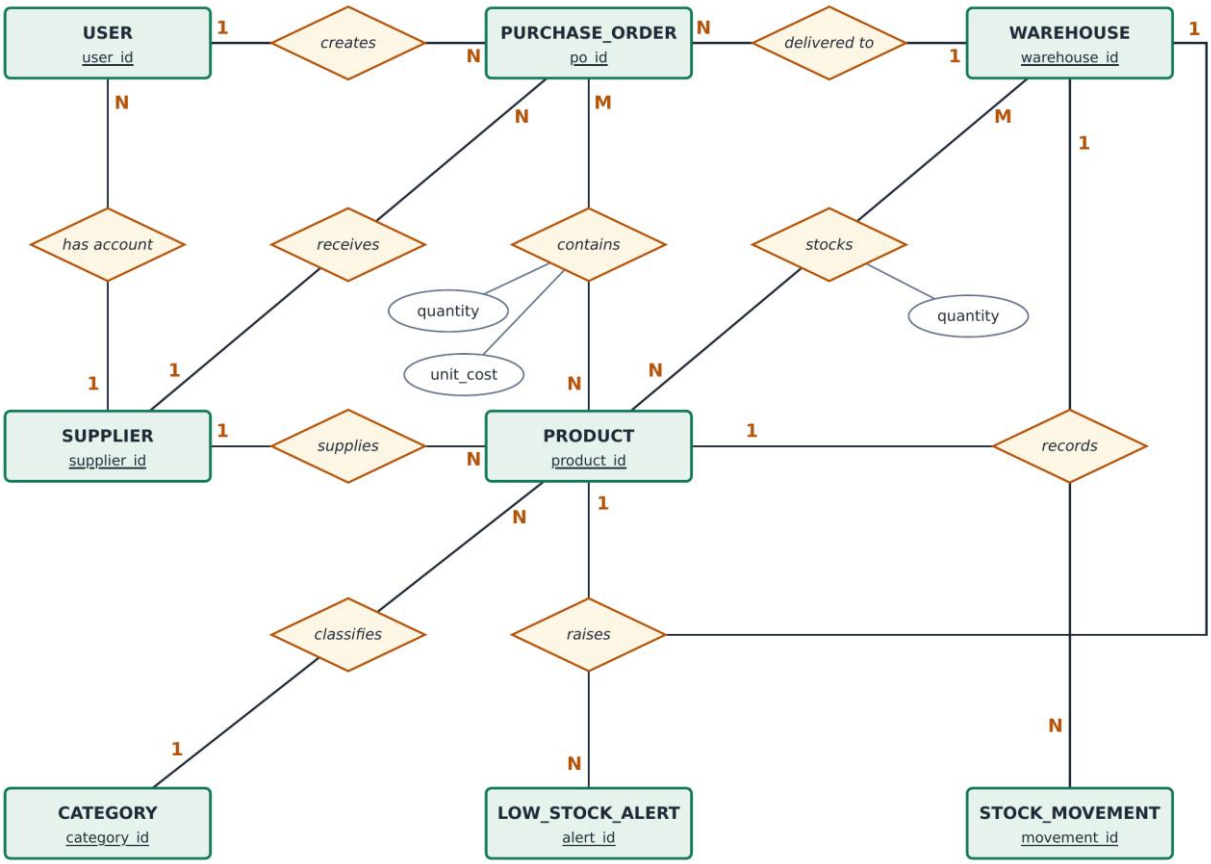


Figure 3.1: ER diagram of the relational part of the system (primary keys underlined)

3.4 Relationships and Cardinalities

Relationship	Cardinality	Description
SUPPLIER *supplies* PRODUCT	1 : N	Each product has one primary supplier.
CATEGORY *classifies* PRODUCT	1 : N	Each product belongs to one category.
SUPPLIER *has account* USER	1 : N (partial)	Only SUPPLIER-role users are linked to a supplier.

Relationship	Cardinality	Description
USER *creates* PURCHASE_ORDER	1 : N	The staff member who placed the order.
SUPPLIER *receives* PURCHASE_ORDER	1 : N	Each order is placed with one supplier.
WAREHOUSE *delivered to* PURCHASE_ORDER	1 : N	Each order is delivered to one warehouse.
PURCHASE_ORDER *contains* PRODUCT	M : N	An order has many products and a product appears in many orders. Attributes: *quantity, unit_cost*.
WAREHOUSE *stocks* PRODUCT	M : N	Attribute: *quantity* on hand.
records STOCK_MOVEMENT	1 : 1 : N	Each movement concerns one product in one warehouse.
raises LOW_STOCK_ALERT	1 : 1 : N	Each alert concerns one product in one warehouse.

The MongoDB documents are not part of the ER model, because they are not relational. Each PRODUCT has at most one `product_specs` document and each SUPPLIER at most one `supplier_documents` document (1 : 0..1). Section 7 describes them.

4. Relational Model

4.1 List of Relations

The ER model maps to **ten tables**. The relations marked **R** are *not* entities in the ER model: they come from M : N relationships and carry the attributes of those relationships.

Relation	Primary key	Foreign keys	Origin in ER model
CATEGORIES	category_id	–	Entity
SUPPLIERS	supplier_id	–	Entity
USERS	user_id	supplier_id → SUPPLIERS (nullable)	Entity
PRODUCTS	product_id	category_id → CATEGORIES; supplier_id → SUPPLIERS	Entity
WAREHOUSES	warehouse_id	–	Entity
PURCHASE_ORDERS	po_id	supplier_id → SUPPLIERS; warehouse_id → WAREHOUSES; created_by → USERS	Entity
INVENTORY (R)	(warehouse_id, product_id)	warehouse_id → WAREHOUSES; product_id → PRODUCTS	**Relationship** *stocks* (M : N), attribute quantity
PO_ITEMS (R)	(po_id, product_id)	po_id → PURCHASE_ORDERS (ON DELETE CASCADE); product_id → PRODUCTS	**Relationship** *contains* (M : N), attributes quantity, unit_cost
STOCK_MOVEMENTS	movement_id	product_id → PRODUCTS; warehouse_id → WAREHOUSES	Entity (ternary relationship *records*)
LOW_STOCK_ALERTS	alert_id	product_id → PRODUCTS; warehouse_id → WAREHOUSES	Entity (ternary relationship *raises*)

4.2 Schema Diagram

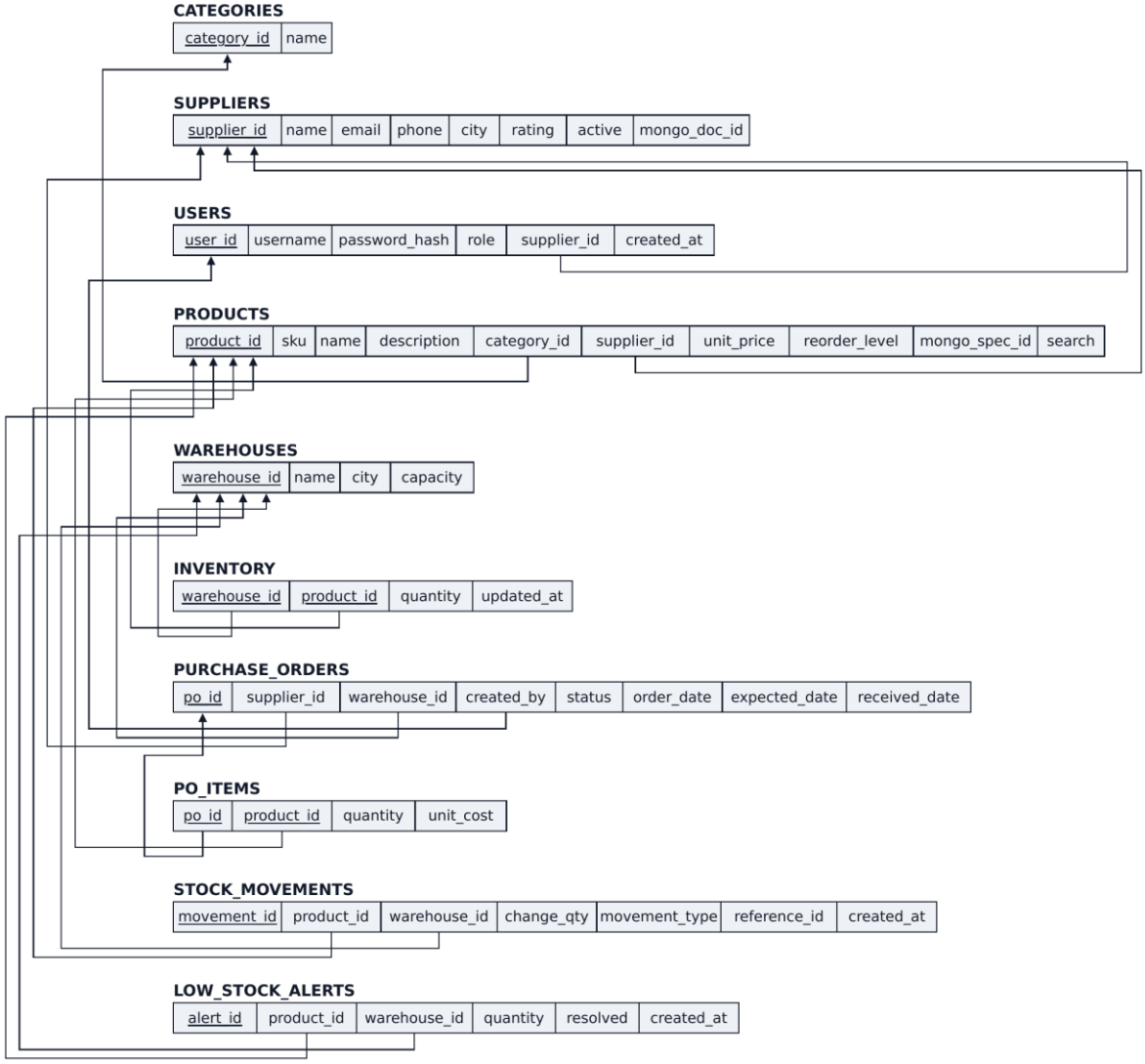


Figure 4.1: Relational database schema for the inventory database. Primary keys are underlined; each arrow goes from a foreign key to the primary key it references.

4.3 Mapping and Normalisation

1. **Strong entities → tables.** Each entity becomes a table with a surrogate primary key (`SERIAL` / `BIGSERIAL`). Natural candidate keys are kept as UNIQUE constraints: `sku`, `username`, supplier `email`, warehouse and category `name`.
2. **1 : N relationships → foreign keys** on the "many" side: `products.category_id`, `products.supplier_id`, `purchase_orders.supplier_id` / `warehouse_id` / `created_by`, `users.supplier_id`.

3. **M : N relationships → junction tables** with composite primary keys. *contains* becomes **PO_ITEMS** (PK `po_id, product_id`) carrying `quantity` and `unit_cost`; *stocks* becomes **INVENTORY** (PK `warehouse_id, product_id`) carrying `quantity`.
4. **Ternary relationships** (*records*, *raises*) become tables with two foreign keys: STOCK_MOVEMENTS and LOW_STOCK_ALERTS.
5. **Partial participation** (only supplier users have a supplier) becomes a nullable `users.supplier_id`, with a CHECK constraint so that `role = 'SUPPLIER'` exactly when `supplier_id` is set.

- **1NF:** all attributes are atomic. Lists such as a product's specifications or a supplier's certificates are not stored in PostgreSQL at all; they are in MongoDB.
- **2NF:** in the two tables with composite keys (INVENTORY, PO_ITEMS), every non-key attribute depends on the whole key. Stock quantity depends on (warehouse, product), and order quantity and cost depend on (order, product).
- **3NF:** there are no transitive dependencies. Supplier and category names are stored once and referenced by key; an order's total value is not stored but computed with `SUM(quantity × unit_cost)`, and stock value is computed in a view.
- **Deliberate exception:** `low_stock_alerts.quantity` is a snapshot of the stock level when the alert was raised. It is historical data, not a duplicate of current inventory.

5. SQL DDL Statements

PostgreSQL 18 · file `db/schema.sql` (DDL). The complete script is in the GitHub repository; the main statements are shown here.

5.1 CREATE, ALTER, DROP

The schema script is re-runnable. It first **drops** and recreates the schema, then **creates** the tables. Because `users` and `suppliers` reference each other in the design, the supplier foreign key and the role rule are added afterwards with **ALTER TABLE**.

```
DROP SCHEMA IF EXISTS public CASCADE;
CREATE SCHEMA public;


CREATE TABLE users (
    user_id        SERIAL PRIMARY KEY,
    username       VARCHAR(50) NOT NULL UNIQUE,
    password_hash  TEXT        NOT NULL,                -- bcrypt hash, never plain text
    role           VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN','MANAGER','SUPPLIER')),
    supplier_id    INT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);


ALTER TABLE users
    ADD CONSTRAINT fk_users_supplier FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id),
    ADD CONSTRAINT chk_supplier_role CHECK ((role = 'SUPPLIER') = (supplier_id IS NOT NULL));


CREATE TABLE products (
    product_id     SERIAL PRIMARY KEY,
    sku            VARCHAR(30) NOT NULL UNIQUE,
    ...
    unit_price     NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    search         tsvector GENERATED ALWAYS AS
                   (to_tsvector('english', coalesce(name,'') || ' ' || coalesce(description,''))) STORED
);


CREATE TABLE purchase_orders (
    ...
    status   VARCHAR(12) NOT NULL DEFAULT 'ORDERED' CHECK (status IN ('ORDERED','RECEIVED','CANCELLED')),
    CHECK (expected_date IS NULL OR expected_date >= order_date),
    CHECK ((status = 'RECEIVED') = (received_date IS NOT NULL))
);
```

5.2 Indexes

Index	Type	Purpose
`idx_products_search`	GIN on tsvector	Full-text product search
`idx_products_category`, `idx_products_supplier`	B-tree	Joins and filters on foreign keys
`idx_po_status`	Composite B-tree	Purchase orders filtered by status and supplier
`idx_movements_product`	Composite, DESC	Latest movements of a product
`idx_alerts_open`	Partial (WHERE resolved = FALSE)	Fast lookup of open alerts only; stays small

5.3 Views

```
CREATE VIEW v_stock_summary AS                        -- total units and value per product
SELECT p.product_id, p.sku, p.name, c.name AS category,
       COALESCE(SUM(i.quantity),0) AS total_stock,
       COALESCE(SUM(i.quantity),0) * p.unit_price AS stock_value
FROM products p JOIN categories c ON c.category_id = p.category_id
LEFT JOIN inventory i ON i.product_id = p.product_id
GROUP BY p.product_id, p.sku, p.name, c.name, p.unit_price;


CREATE VIEW v_reorder_suggestions AS                  -- what to reorder, how much, from whom
SELECT i.warehouse_id, w.name AS warehouse, p.product_id, p.sku, p.name, i.quantity, p.reorder_level,
       (p.reorder_level * 2 - i.quantity) AS suggested_order_qty, s.supplier_id, s.name AS supplier
FROM inventory i
JOIN products p ON p.product_id = i.product_id JOIN warehouses w ON w.warehouse_id = i.warehouse_id
JOIN suppliers s ON s.supplier_id = p.supplier_id
WHERE i.quantity <= p.reorder_level;
```

5.4 Triggers

Trigger 1: warehouse capacity (BEFORE INSERT OR UPDATE on inventory)

Before any inventory row changes, the trigger adds up the warehouse's other stock and rejects the change if the total would exceed capacity. Because it runs inside the database, no application path can overfill a warehouse.

```
CREATE FUNCTION check_warehouse_capacity() RETURNS trigger AS $$
DECLARE v_cap INT; v_used INT;
BEGIN
    SELECT capacity INTO v_cap FROM warehouses WHERE warehouse_id = NEW.warehouse_id;
    SELECT COALESCE(SUM(quantity),0) INTO v_used
      FROM inventory WHERE warehouse_id = NEW.warehouse_id AND product_id <> NEW.product_id;
    IF v_used + NEW.quantity > v_cap THEN
        RAISE EXCEPTION 'Warehouse % capacity exceeded (% + % > %)', NEW.warehouse_id, v_used,
NEW.quantity, v_cap;
    END IF;
    NEW.updated_at := now();
    RETURN NEW;
END $$ LANGUAGE plpgsql;


CREATE TRIGGER trg_capacity BEFORE INSERT OR UPDATE ON inventory
FOR EACH ROW EXECUTE FUNCTION check_warehouse_capacity();
```

Trigger 2: low-stock alerts (AFTER INSERT OR UPDATE OF quantity on inventory)

When stock crosses down to the reorder level, an alert is inserted. When it rises above the level again, open alerts for that product and warehouse are marked resolved. The `TG_OP = 'INSERT'` check also covers a transfer that creates a new, already-low inventory row.

```
CREATE FUNCTION raise_low_stock_alert() RETURNS trigger AS $$
DECLARE v_level INT;
BEGIN
    SELECT reorder_level INTO v_level FROM products WHERE product_id = NEW.product_id;
    IF NEW.quantity <= v_level AND (TG_OP = 'INSERT' OR OLD.quantity > v_level) THEN
        INSERT INTO low_stock_alerts(product_id, warehouse_id, quantity)
        VALUES (NEW.product_id, NEW.warehouse_id, NEW.quantity);
    ELSIF NEW.quantity > v_level THEN
        UPDATE low_stock_alerts SET resolved = TRUE
         WHERE product_id = NEW.product_id AND warehouse_id = NEW.warehouse_id AND NOT resolved;
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;


CREATE TRIGGER trg_low_stock AFTER INSERT OR UPDATE OF quantity ON inventory
```

```
FOR EACH ROW EXECUTE FUNCTION raise_low_stock_alert();
```

5.5 Stored Procedures

Both procedures run as a single transaction: if any statement fails (including a trigger raising an exception), every change is rolled back.

receive_purchase_order(po_id)

Locks the order row with `SELECT … FOR UPDATE`, so two users clicking "Receive" at the same moment cannot both succeed. It then adds each item to inventory (an upsert with `ON CONFLICT`), logs a `PO_RECEIPT` movement and marks the order RECEIVED.

```
CREATE PROCEDURE receive_purchase_order(p_po_id INT) LANGUAGE plpgsql AS $$
DECLARE v_status TEXT; v_wh INT; r RECORD;
BEGIN
    SELECT status, warehouse_id INTO v_status, v_wh
      FROM purchase_orders WHERE po_id = p_po_id FOR UPDATE;          -- lock the PO row
    IF NOT FOUND THEN RAISE EXCEPTION 'PO % not found', p_po_id; END IF;
    IF v_status <> 'ORDERED' THEN RAISE EXCEPTION 'PO % is already %', p_po_id, v_status; END IF;

    FOR r IN SELECT product_id, quantity FROM po_items WHERE po_id = p_po_id LOOP
        INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES (v_wh, r.product_id, r.quantity)
        ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity +
EXCLUDED.quantity;
        INSERT INTO stock_movements(product_id, warehouse_id, change_qty, movement_type, reference_id)
        VALUES (r.product_id, v_wh, r.quantity, 'PO_RECEIPT', p_po_id);
    END LOOP;
    UPDATE purchase_orders SET status = 'RECEIVED', received_date = CURRENT_DATE WHERE po_id = p_po_id;
END $$;
```

transfer_stock(product, from, to, qty)

Locks both inventory rows **in a fixed order** (`ORDER BY warehouse_id`). Two transfers in opposite directions therefore always lock in the same sequence and cannot deadlock. The procedure then checks stock, moves it and logs both sides. A capacity violation in the destination raises an exception from the trigger and rolls back the whole transfer.

```
CREATE PROCEDURE transfer_stock(p_product INT, p_from INT, p_to INT, p_qty INT) LANGUAGE plpgsql AS $$
DECLARE v_have INT;
BEGIN
    IF p_from = p_to THEN RAISE EXCEPTION 'Source and destination must differ'; END IF;
    IF p_qty <= 0 THEN RAISE EXCEPTION 'Quantity must be positive'; END IF;
    PERFORM 1 FROM inventory WHERE product_id = p_product AND warehouse_id IN (p_from, p_to)
     ORDER BY warehouse_id FOR UPDATE;                       -- fixed lock order: no deadlocks
    SELECT quantity INTO v_have FROM inventory WHERE product_id = p_product AND warehouse_id = p_from;
    IF v_have IS NULL OR v_have < p_qty THEN
        RAISE EXCEPTION 'Insufficient stock: have %, need %', COALESCE(v_have,0), p_qty;
    END IF;
    UPDATE inventory SET quantity = quantity - p_qty WHERE product_id = p_product AND warehouse_id =
p_from;
    INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES (p_to, p_product, p_qty)
    ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity +
EXCLUDED.quantity;
    INSERT INTO stock_movements(product_id, warehouse_id, change_qty, movement_type)
    VALUES (p_product, p_from, -p_qty, 'TRANSFER_OUT'), (p_product, p_to, p_qty, 'TRANSFER_IN');
END $$;
```

6. SQL DML Statements

PostgreSQL 18 · file `db/seed_and_test.sql` (sample data, DML and tests) and the queries used by the API routes.

6.1 INSERT, UPDATE, DELETE

```
-- INSERT: sample data (db/seed_and_test.sql)
INSERT INTO suppliers(name,email,city,rating) VALUES ('Acme Foods','a@acme.com','Bengaluru',4.5),('Globex
Parts','g@globex.com','Chennai',3.8);
INSERT INTO categories(name) VALUES ('Grocery'),('Electronics');
INSERT INTO products(sku,name,description,category_id,supplier_id,unit_price,reorder_level)
VALUES ('G-001','Basmati Rice 5kg','Long grain aged rice',1,1,450,20),('E-001','USB-C Cable','Braided fast
charging cable',2,2,299,15);
INSERT INTO warehouses(name,city,capacity) VALUES ('WH-Blr','Bengaluru',500),('WH-Che','Chennai',40),('WH-
Hyd','Hyderabad',500);
INSERT INTO purchase_orders(supplier_id,warehouse_id,created_by) VALUES (1,1,1);
INSERT INTO po_items VALUES (1,1,100,400);


-- INSERT with RETURNING (add a supplier; email format, uniqueness and rating are checked by the DB)
INSERT INTO suppliers (name, email, phone, city, rating, active)
VALUES ($1, $2, $3, $4, COALESCE($5::numeric, 0), true) RETURNING supplier_id, name;


-- UPDATE with COALESCE (partial edit / soft delete of a supplier)
UPDATE suppliers SET name = COALESCE($1, name), email = COALESCE($2, email), active = COALESCE($6::boolean,
active)
WHERE supplier_id = $7;


-- UPSERT (INSERT ... ON CONFLICT) inside the procedures
INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES ($1, $2, $3)
ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity + EXCLUDED.quantity;


-- Soft delete: the application never hard-deletes business records, so history is kept
UPDATE suppliers SET active = false WHERE supplier_id = $1;                        -- deactivate a supplier
UPDATE purchase_orders SET status = 'CANCELLED' WHERE po_id = $1 AND status = 'ORDERED'; -- cancel a PO


-- Hard DELETE (SQL level): po_items rows go automatically through ON DELETE CASCADE
DELETE FROM purchase_orders WHERE po_id = $1 AND status = 'CANCELLED';
```

Business records are deactivated or cancelled rather than deleted, so stock history and order history stay intact for auditing. Physical deletion is supported at the SQL level through `ON DELETE CASCADE` on `po_items`.

6.2 JOIN + GROUP BY + aggregate: stock value per category

```
SELECT v.category, SUM(v.stock_value) AS stock_value, SUM(v.total_stock)::int AS units
FROM v_stock_summary v
JOIN products p ON p.product_id = v.product_id
WHERE ($1::int IS NULL OR p.supplier_id = $1)          -- suppliers see only their own data
GROUP BY v.category
ORDER BY stock_value DESC;
```

6.3 LEFT JOIN + computed ratio: warehouse utilisation

```
SELECT w.warehouse_id, w.name, w.capacity,
       COALESCE(SUM(i.quantity), 0)::int AS used,
       ROUND(100.0 * COALESCE(SUM(i.quantity), 0) / w.capacity, 1) AS utilisation_pct
FROM warehouses w
LEFT JOIN inventory i ON i.warehouse_id = w.warehouse_id     -- empty warehouses still appear
GROUP BY w.warehouse_id, w.name, w.capacity
ORDER BY utilisation_pct DESC;
```

6.4 GROUP BY … HAVING: suppliers with more than N ordered units

```
SELECT s.supplier_id, s.name, SUM(i.quantity)::int AS ordered_units, COUNT(DISTINCT po.po_id)::int AS
orders
FROM suppliers s
JOIN purchase_orders po ON po.supplier_id = s.supplier_id
JOIN po_items i         ON i.po_id        = po.po_id
WHERE po.status <> 'CANCELLED'
GROUP BY s.supplier_id, s.name
HAVING SUM(i.quantity) > $2
ORDER BY ordered_units DESC;
```

6.5 Correlated subquery (NOT EXISTS): products never ordered

```
SELECT p.product_id, p.sku, p.name, c.name AS category
FROM products p JOIN categories c ON c.category_id = p.category_id
WHERE NOT EXISTS (SELECT 1 FROM po_items i WHERE i.product_id = p.product_id)
ORDER BY p.name;
```

6.6 Scalar subqueries: dashboard totals

```
SELECT (SELECT COUNT(*) FROM products)                                    AS products,
       (SELECT COUNT(*) FROM warehouses)                                  AS warehouses,
       (SELECT COALESCE(SUM(stock_value), 0) FROM v_stock_summary)        AS stock_value,
       (SELECT COUNT(*) FROM low_stock_alerts WHERE resolved = false)     AS open_alerts;
```

6.7 Purchase-order list with totals

```
SELECT po.po_id, po.status, s.name AS supplier, w.name AS warehouse,
       COUNT(i.product_id) AS item_count,
       COALESCE(SUM(i.quantity * i.unit_cost), 0) AS total_value
FROM purchase_orders po
JOIN suppliers s ON s.supplier_id = po.supplier_id
JOIN warehouses w ON w.warehouse_id = po.warehouse_id
LEFT JOIN po_items i ON i.po_id = po.po_id
WHERE po.status = $1
GROUP BY po.po_id, s.name, w.name
ORDER BY po.po_id DESC;
```

6.8 Recent stock movements (ORDER BY … LIMIT)

```
SELECT m.movement_id, p.sku, w.name AS warehouse, m.change_qty, m.movement_type, m.created_at
FROM stock_movements m
JOIN products p ON p.product_id = m.product_id JOIN warehouses w ON w.warehouse_id = m.warehouse_id
ORDER BY m.created_at DESC, m.movement_id DESC LIMIT 10;
```

6.9 Transactions

The two stored procedures (Section 5.5) are called with `CALL receive_purchase_order($1)` and `CALL transfer_stock($1, $2, $3, $4)`. Creating a purchase order is an **application-level transaction**: it writes one header row and several item rows. The API takes one dedicated connection from the pool and wraps them in `BEGIN … COMMIT`, with `ROLLBACK` on any error, so an order is never saved without its items.

```
const client = await pool.connect();
try {
  await client.query("BEGIN");
  const po = await client.query(`INSERT INTO purchase_orders (supplier_id, warehouse_id, created_by, status,
order_date, expected_date)
                                 VALUES ($1, $2, $3, 'ORDERED', CURRENT_DATE, $4) RETURNING po_id`, [...]);
  for (const it of items)
```

```
    await client.query(`INSERT INTO po_items (po_id, product_id, quantity, unit_cost) VALUES ($1, $2, $3, $4)`,
[...]);
  await client.query("COMMIT");
} catch (e) {
  await client.query("ROLLBACK");                  // nothing is half-saved
} finally { client.release(); }
```

6.10 Full-text search

The `search` column is a **generated** `tsvector` built from the name and description, so it never goes out of date. A GIN index makes matching fast, and results are ranked by relevance. The `english` configuration applies stemming, so *charging cable* matches "*Braided fast charging cable*".

```
SELECT p.sku, p.name, ts_rank(p.search, plainto_tsquery('english', $1)) AS relevance
FROM products p
WHERE p.search @@ plainto_tsquery('english', $1)
ORDER BY relevance DESC;
```

6.11 Advanced features: summary

The brief requires at least four advanced SQL features and one NoSQL feature. This project implements the following:

#	Feature	Where
1	Transactions	`receive_purchase_order`, `transfer_stock` (row locks, fixed lock order), PO creation with BEGIN/COMMIT/ROLLBACK
2	Stored procedures	`receive_purchase_order`, `transfer_stock` (PL/pgSQL)
3	Triggers	`trg_capacity` (BEFORE), `trg_low_stock` (AFTER)
4	Views	`v_stock_summary`, `v_reorder_suggestions`
5	Indexing	GIN, composite, descending and partial indexes
6	Full-text search	Generated tsvector, `plainto_tsquery`, `ts_rank`
7	NoSQL aggregation	4 MongoDB pipelines (Section 7.5)
8	NoSQL schema validation	`$jsonSchema` with per-category `oneOf` rules

7. NoSQL Database Commands (MongoDB)

MongoDB Atlas · database `supply_chain` · files `mongo/setup.mjs`, `web/lib/pipelines.js`. No vector database is used (see Section 2.3).

7.1 Data model

Two collections, each with one document per PostgreSQL row it describes. Embedding is used throughout: certificates, contacts and feedback are arrays inside the supplier's document, because they are always read together with the supplier and never on their own.

```
// product_specs: Grocery
{
  product_id: 1, sku: "G-001",
  category: "Grocery",
  specs: {
    net_weight_kg: 5, grain: "Long grain",
    aged_months: 12, shelf_life_days: 365,
    storage: "Cool, dry place",
    fssai_license: "10012345000123",
    nutrition_per_100g: {
      energy_kcal: 350, protein_g: 7.5,
      carbs_g: 78 }
  },
  images: ["rice-front.jpg"]
}
```

```
// product_specs: Electronics
{
  product_id: 2, sku: "E-001",
  category: "Electronics",
  specs: {
    connector: "USB-C to USB-C",
    length_m: 1.5, max_wattage: 60,
    data_rate_gbps: 0.48, braided: true,
    warranty_months: 12,
    certifications: ["BIS", "RoHS"]
  },
  images: ["cable-coil.jpg"]
}
```

```
// supplier_documents
{
  supplier_id: 2,
  certifications: [ { type: "BIS", number: "R-41012345", valid_until: ISODate("2026-11-21") },
                    { type: "GST", number: "33ABCDE1234F1Z5", valid_until: ISODate("2029-07-23") } ],
  contacts: [ { name: "S. Iyer", role: "Sales", email: "sales@globex.com" } ],
  feedback: [ { po_id: 4, rating: 3, issues: ["late_delivery"], comment: "Late by a week", date:
ISODate(...) } ]
}
```

7.2 Creating collections with validators

Both collections have a `$jsonSchema` validator with `validationAction: "error"`, so invalid documents are rejected by the database. The specs validator uses `oneOf`, so the **shape is flexible but still enforced per category**:

```
validator: { $jsonSchema: {
  bsonType: "object", required: ["product_id", "sku", "category", "specs"],
  properties: { product_id: { bsonType: "int", minimum: 1 }, ... },
  oneOf: [
    { properties: { category: { enum: ["Grocery"] },
                    specs: { required: ["net_weight_kg", "shelf_life_days", "storage"] } } },
```

```
    { properties: { category: { enum: ["Electronics"] },
                    specs: { required: ["warranty_months", "certifications"] } } }
  ] } }
// supplier_documents: feedback.rating must be { bsonType: "int", minimum: 1, maximum: 5 }
```

When MongoDB rejects a document (error code 121), the API returns the validator's message to the UI, in the same way PostgreSQL constraint errors are shown.

7.3 Indexes

```
db.product_specs.createIndex({ product_id: 1 }, { unique: true })
db.product_specs.createIndex({ sku: 1 }, { unique: true })
db.product_specs.createIndex({ category: 1 })
db.supplier_documents.createIndex({ supplier_id: 1 }, { unique: true })
db.supplier_documents.createIndex({ "certifications.valid_until": 1 })      // expiring-certificate
queries
db.supplier_documents.createIndex({ "feedback.issues": 1 })                 // multikey index on issue
tags
```

Collection	Index	Purpose
product_specs	`{ product_id: 1 }` unique	Look up specs from a PostgreSQL product
product_specs	`{ sku: 1 }` unique	Look up by SKU
product_specs	`{ category: 1 }`	Filter by category
supplier_documents	`{ supplier_id: 1 }` unique	One document per supplier
supplier_documents	`{ "certifications.valid_until": 1 }`	Expiring-certificate queries
supplier_documents	`{ "feedback.issues": 1 }` multikey	Queries by issue tag

7.4 Insert, find and update

```
db.product_specs.insertMany([
  { product_id: 1, sku: "G-001", category: "Grocery",
    specs: { net_weight_kg: 5, grain: "Long grain", shelf_life_days: 365, storage: "Cool, dry place" } },
  { product_id: 2, sku: "E-001", category: "Electronics",
    specs: { connector: "USB-C to USB-C", max_wattage: 60, warranty_months: 12, certifications: ["BIS",
"RoHS"] } }
])
```

```
db.product_specs.findOne({ product_id: 1 })                                  // specs for one product
db.product_specs.find({ category: "Electronics", "specs.max_wattage": { $gte: 45 } }) // query on a
nested field
db.supplier_documents.updateOne(                                             // add feedback, create
the doc if missing
  { supplier_id: 2 },
  { $push: { feedback: { po_id: 3, rating: 4, issues: ["late_delivery"], date: new Date() } },
    $setOnInsert: { certifications: [], contacts: [] } },
  { upsert: true })
```

7.5 Aggregation pipelines

Pipeline 1: supplier scorecard

```
[ { $unwind: "$feedback" },
  { $group: { _id: "$supplier_id",
              avg_rating:  { $avg: "$feedback.rating" },
              reviews:     { $sum: 1 },
              issue_count: { $sum: { $size: { $ifNull: ["$feedback.issues", []] } } },
              last_feedback: { $max: "$feedback.date" } } },
  { $project: { _id: 0, supplier_id: "$_id", avg_rating: { $round: ["$avg_rating", 2] },
                reviews: 1, issue_count: 1, last_feedback: 1 } },
```

```
  { $sort: { avg_rating: 1 } } ]                        // worst suppliers first
```

Pipeline 2: most common delivery issues

```
[ { $unwind: "$feedback" }, { $unwind: "$feedback.issues" },
  { $group: { _id: "$feedback.issues", occurrences: { $sum: 1 }, suppliers: { $addToSet:
"$supplier_id" } } },
  { $sort: { occurrences: -1 } } ]
```

Pipeline 3: certificates expiring within N days

```
[ { $unwind: "$certifications" },
  { $match: { "certifications.valid_until": { $lte: new Date(Date.now() + days * 86400000) } } },
  { $project: { supplier_id: 1, type: "$certifications.type", number: "$certifications.number",
                days_left: { $dateDiff: { startDate: "$$NOW", endDate: "$certifications.valid_until",
unit: "day" } } } },
  { $sort: { days_left: 1 } } ]
```

Pipeline 4: specification fields by category

```
[ { $project: { category: 1, product_id: 1,
                fields: { $map: { input: { $objectToArray: "$specs" }, as: "f", in: "$$f.k" } } } },
  { $unwind: "$fields" },
  { $group: { _id: "$category", spec_fields: { $addToSet: "$fields" }, products: { $addToSet:
"$product_id" } } },
  { $project: { category: "$_id", products: 1,
                spec_fields: { $sortArray: { input: "$spec_fields", sortBy: 1 } } } } ]
```

7.6 Retrieval strategy

1. **Product page:** the product row and per-warehouse stock come from PostgreSQL. Then `product_specs.findOne({ product_id })` fetches the specs, using the unique index.
2. **Supplier page:** the supplier row and its RECEIVED purchase orders come from PostgreSQL. The supplier document and a per-supplier scorecard (pipeline 1 with an extra first `$match`) come from MongoDB. The page then marks which orders already have feedback.
3. **Recording feedback:** the API checks in PostgreSQL that the order belongs to the supplier and is RECEIVED. It then `$push`es into `feedback` with `upsert: true`, recomputes the average and writes it back to `suppliers.rating`. If the supplier had no document yet, the new ObjectId is stored in `suppliers.mongo_doc_id`.
4. **Resilience:** if MongoDB cannot be reached, product and supplier pages still show their PostgreSQL data with a warning.

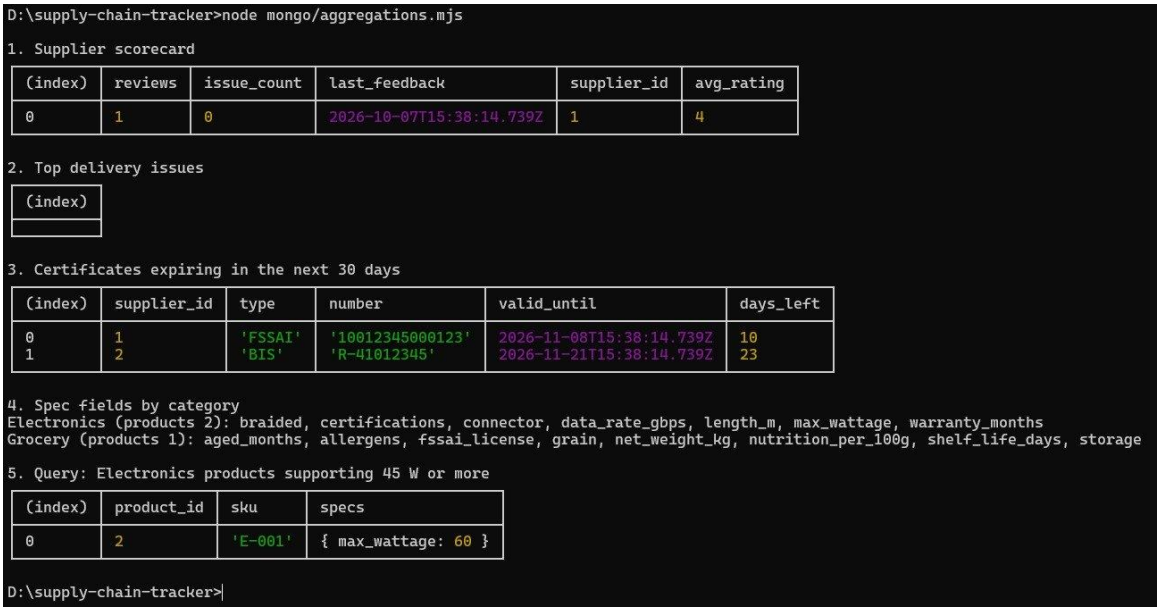


Figure 7.1: Output of the aggregation pipelines and the nested-field query (`node mongo/aggregations.mjs`)

8. Results

8.1 Resulting tables' screenshots (psql test run)

The scripts `db/schema.sql` and `db/seed_and_test.sql` were run with `psql` against the database `inventory_db`. The output below shows the schema being created, the seed data and procedure calls, the automatic alert, the three expected errors, and the resulting tables and views. The test numbers (T1–T18) refer to the table in Section 8.3.



Figure 8.1: Running schema.sql: DROP SCHEMA … CASCADE, then CREATE / ALTER TABLE statements (T1)

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE VIEW
CREATE VIEW
CREATE PROCEDURE
CREATE PROCEDURE

C:\Program Files\PostgreSQL\18\bin>psql -U postgres -d inventory_db -f "C:\Users\rahul\OneDrive\Documents\Rahul\year3\DBMSLab\DBMS_Mi
ni_Project\DBMS-Mini_Project\db\seed_and_test.sql"
Password for user postgres:

INSERT 0 2
INSERT 0 2
INSERT 0 1
INSERT 0 2
INSERT 0 3
INSERT 0 1
INSERT 0 1
CALL
CALL
 warehouse_id | product_id | quantity
--------------+------------+----------
            1 |          1 |       15
            3 |          1 |       85
(2 rows)
```

Figure 8.2: Indexes, functions, triggers, views and procedures created; seed data inserted; inventory after the receive and transfer (T1–T3)

```
 product_id | warehouse_id | quantity | resolved
------------+--------------+----------+----------
          1 |            1 |       15 | f
(1 row)


 warehouse_id | warehouse | product_id |  sku  |       name       | quantity | reorder_level | suggested_order_qty | supplier_id |  s
upplier
--------------+-----------+------------+-------+------------------+----------+---------------+---------------------+-------------+---
---------
            1 | WH-Blr    |          1 | G-001 | Basmati Rice 5kg |       15 |            20 |                  25 |           1 | Ac
me Foods
(1 row)


CALL
 product_id | warehouse_id | quantity | resolved
------------+--------------+----------+----------
          1 |            1 |       15 | t
(1 row)


--- expected errors ---
psql:C:/Users/rahul/OneDrive/Documents/Rahul/year3/DBMSLab/DBMS_Mini_Project/DBMS-Mini_Project/db/seed_and_test.sql:17: ERROR:  Insuf
ficient stock: have 45, need 500
CONTEXT:  PL/pgSQL function transfer_stock(integer,integer,integer,integer) line 13 at RAISE
psql:C:/Users/rahul/OneDrive/Documents/Rahul/year3/DBMSLab/DBMS_Mini_Project/DBMS-Mini_Project/db/seed_and_test.sql:18: ERROR:  Wareh
ouse 2 capacity exceeded (0 + 45 > 40)
CONTEXT:  PL/pgSQL function check_warehouse_capacity() line 9 at RAISE
SQL statement "INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES (p_to, p_product, p_qty)
    ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity + EXCLUDED.quantity"
PL/pgSQL function transfer_stock(integer,integer,integer,integer) line 18 at SQL statement
psql:C:/Users/rahul/OneDrive/Documents/Rahul/year3/DBMSLab/DBMS_Mini_Project/DBMS-Mini_Project/db/seed_and_test.sql:19: ERROR:  PO 1
is already RECEIVED
CONTEXT:  PL/pgSQL function receive_purchase_order(integer) line 7 at RAISE
--- movements / views / FTS ---
```

Figure 8.3: Alert created (resolved = f), reorder suggestion from the view, alert resolved (resolved = t), and the three expected errors (T3–T8)

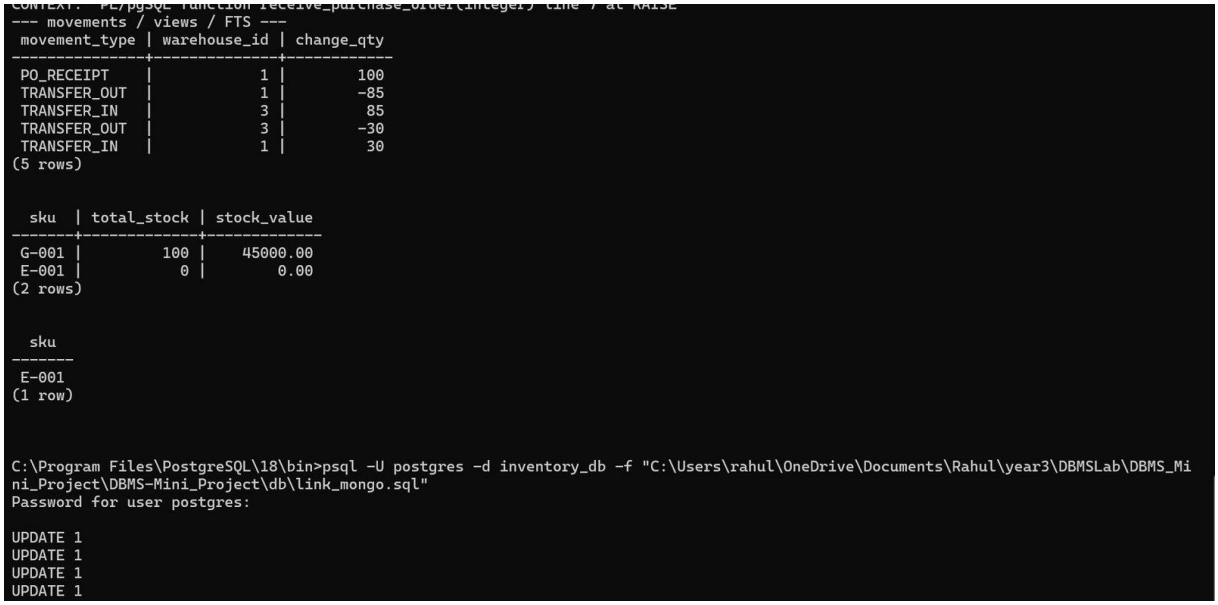


Figure 8.4: Stock movement log, v_stock_summary and the full-text search result; link_mongo.sql updates (T9, T10)

8.2 Validation and error handling (demo)

Database rules are visible in the interface: when PostgreSQL rejects an operation, the application shows the database's own message in a red alert.

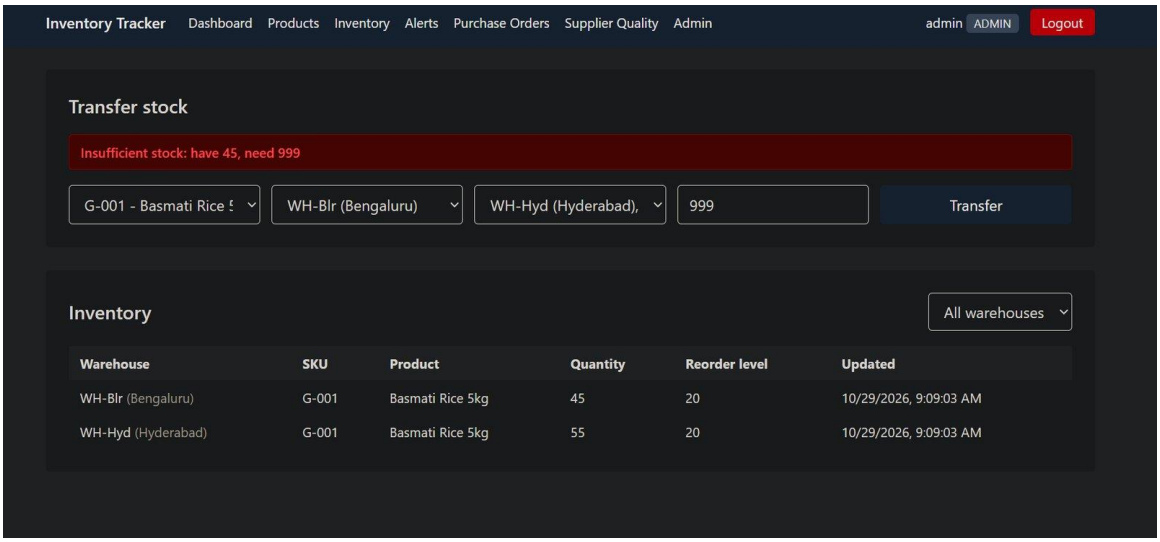


Figure 8.5: Insufficient stock rejected by the transfer_stock procedure (T11)

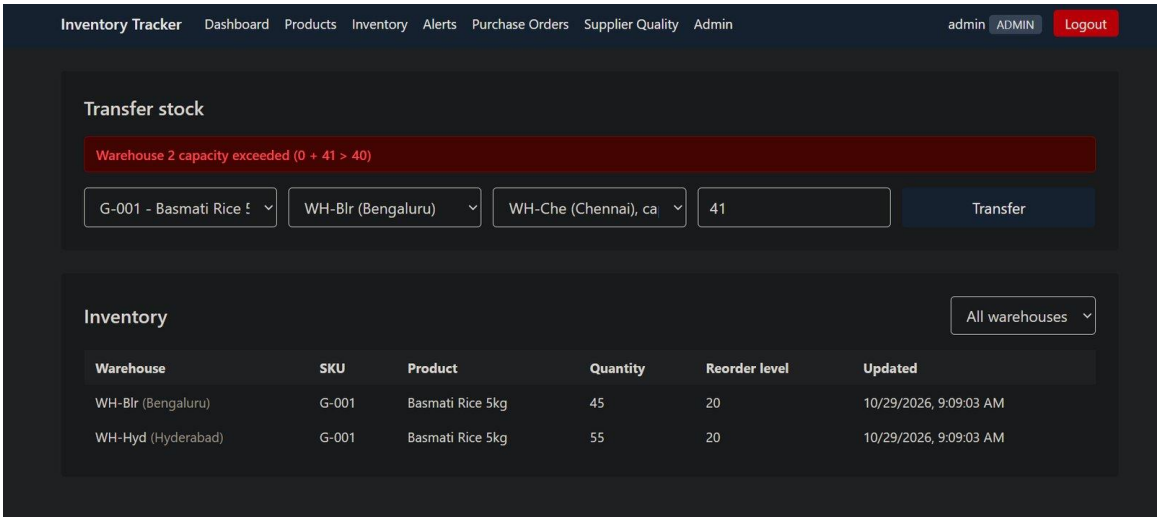


Figure 8.6: Warehouse capacity enforced by the trg_capacity trigger (T12)

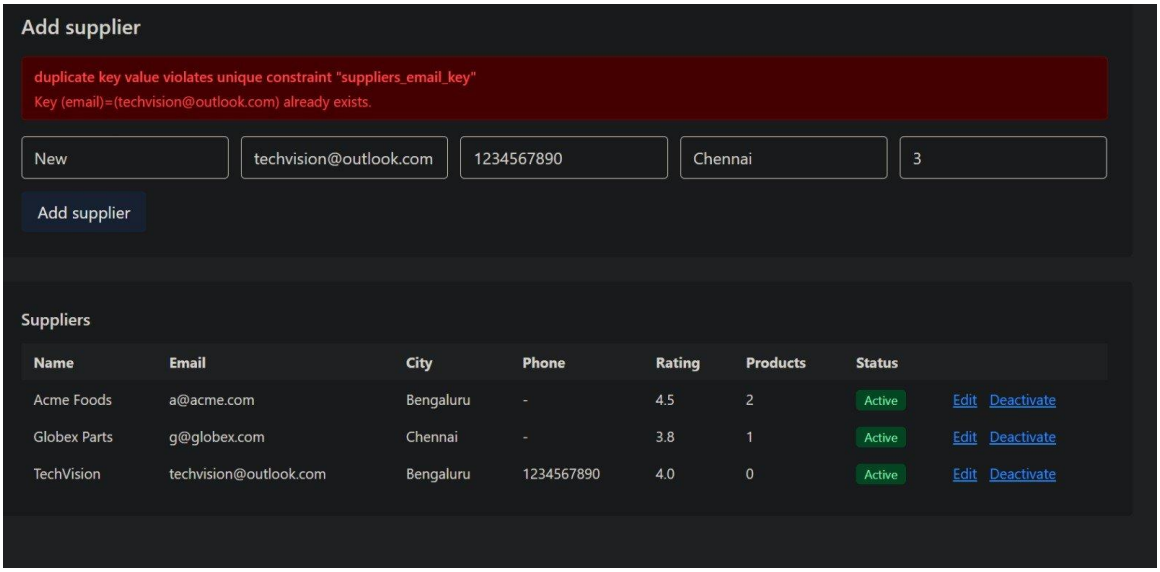


Figure 8.7: Duplicate supplier email rejected by the UNIQUE constraint (T13)

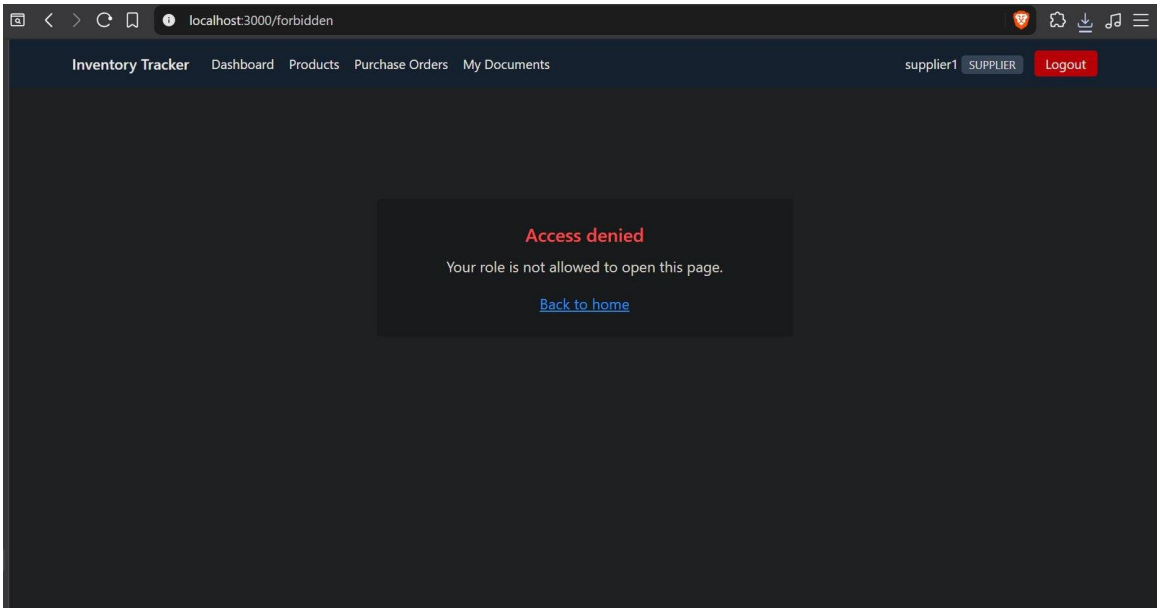


Figure 8.8: SUPPLIER role blocked from the admin area, with a reduced navigation menu (T14)

8.3 Test cases and results

ID	Test	Input / action	Expected result	Result
T1	Schema creation	Run `schema.sql`	10 tables, 6 indexes, 2 triggers, 2 views, 2 procedures created	**Pass**
T2	Receive PO (procedure)	`CALL receive_purchase_order(1)`	100 units added; PO_RECEIPT logged; status RECEIVED	**Pass**
T3	Low-stock alert (trigger)	Transfer 85 of 100 units out of WH-Blr (reorder level 20)	Stock 15; alert created with resolved = f	**Pass**
T4	Reorder view	`SELECT * FROM v_reorder_suggestions`	G-001 at WH-Blr, suggested quantity 25	**Pass**
T5	Alert auto-resolve (trigger)	Transfer 30 units back to WH-Blr	Same alert now resolved = t	**Pass**
T6	Insufficient stock	Transfer 500 units (only 45 available)	Error "Insufficient stock: have 45, need 500"; rollback	**Pass**
T7	Capacity exceeded (trigger)	Transfer 45 units into WH-Che (capacity 40)	Error "Warehouse 2 capacity exceeded (0 + 45 > 40)"; whole transfer rolled back	**Pass**
T8	Double receive (row lock + status check)	`CALL receive_purchase_order(1)` again	Error "PO 1 is already RECEIVED"	**Pass**
T9	Movement log and view	Query stock_movements, v_stock_summary	5 movements; G-001 total 100, value 45 000.00	**Pass**
T10	Full-text search	Search "charging cable" (psql and UI)	Returns E-001	**Pass**
T11	UI: insufficient stock	Transfer 999 units of G-001 from WH-Blr	Red alert with the procedure's message	**Pass**
T12	UI: capacity exceeded	Transfer 41 units to WH-Che	"Warehouse 2 capacity exceeded (0 + 41 > 40)"	**Pass**

ID	Test	Input / action	Expected result	Result
T13	UI: UNIQUE constraint	Add a supplier with an existing email	"duplicate key value violates unique constraint suppliers_email_key"	**Pass**
T14	Role-based access	SUPPLIER user opens `/admin`	Redirected to "Access denied"; supplier menu shows only allowed pages	**Pass**
T15	MongoDB aggregations	`node mongo/aggregations.mjs`	Scorecard, issues, expiring certificates (days_left), spec fields per category, nested-field query	**Pass**
T16	Feedback rules	Feedback for an ORDERED PO, another supplier's PO, a duplicate PO, an unknown tag	Each rejected with HTTP 400 and a clear message	**Pass**
T17	MongoDB validator	Feedback with rating 7 or 4.5 (sent directly to the API)	Rejected by `$jsonSchema` (error 121) and shown as a validation error	**Pass**
T18	Upsert + link	Add a certificate for a supplier with no MongoDB document	Document created; ObjectId written to suppliers.mongo_doc_id	**Pass**

T1–T10: `db/seed_and_test.sql` (Section 8.1). T11–T14: browser (Section 8.2). T15: Figure 7.1. T16–T18: API tests during development of the supplier-quality module.

9. Next.js: Project Tree Structure

Repository layout (generated from the files tracked in GitHub).

The Next.js app uses the **App Router**: every folder under `web/app` is a URL. A `page.js` file is a page the user sees, a `route.js` file under `app/api` is a backend API handler, and a folder in square brackets such as `[id]` is a dynamic segment (e.g. `/products/3`). Database access lives only in `web/lib` and the API routes; pages never touch a database directly.

```
DBMS-Mini_Project/
├── db/                                 # PostgreSQL scripts
│   ├── link_mongo.sql                  # stores MongoDB ObjectIds in PostgreSQL
│   ├── schema.sql                      # DDL: tables, constraints, indexes, triggers, views, procedures
│   └── seed_and_test.sql               # sample data + tests (DML, CALLs, expected errors)
├── docs/                               # report, stock report, screenshots
│   ├── report/
│   │   └── Stock_Report.pdf
│   └── DBMS_Project_Report.pdf
├── mongo/                              # MongoDB Atlas scripts
│   ├── aggregations.mjs                # aggregation pipelines (CLI demo)
│   └── setup.mjs                       # collections, $jsonSchema validators, indexes, sample docs
├── web/                                # Next.js 16 application
│   ├── app/                            # App Router: every folder is a URL
│   │   ├── admin/
│   │   │   ├── suppliers/
│   │   │   │   └── page.js
│   │   │   ├── users/
│   │   │   │   └── page.js
│   │   │   ├── warehouses/
│   │   │   │   └── page.js
│   │   │   └── page.js
│   │   ├── alerts/
│   │   │   └── page.js
│   │   ├── api/                        # backend API (route.js = handler)
│   │   │   ├── alerts/
│   │   │   │   └── route.js
│   │   │   ├── auth/
│   │   │   │   ├── login/
│   │   │   │   │   └── route.js        # bcrypt check, JWT cookie
│   │   │   │   ├── logout/
│   │   │   │   │   └── route.js
│   │   │   │   └── me/
│   │   │   │       └── route.js
│   │   │   ├── dashboard/
│   │   │   │   └── route.js            # GROUP BY, HAVING, NOT EXISTS queries
│   │   │   ├── health/
│   │   │   │   └── route.js
│   │   │   ├── inventory/
│   │   │   │   └── route.js
│   │   │   ├── lookups/
│   │   │   │   └── route.js
│   │   │   ├── products/
│   │   │   │   ├── [id]/
│   │   │   │   │   └── route.js
│   │   │   │   └── route.js            # full-text search
│   │   │   ├── purchase-orders/
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── cancel/
│   │   │   │   │   │   └── route.js
│   │   │   │   │   ├── receive/
│   │   │   │   │   │   └── route.js    # CALL receive_purchase_order(...)
│   │   │   │   │   └── route.js
│   │   │   │   └── route.js            # list + create (BEGIN/COMMIT)
│   │   │   ├── reorder/
│   │   │   │   └── route.js
│   │   │   ├── supplier-quality/
│   │   │   │   └── route.js            # MongoDB pipelines + PostgreSQL names
│   │   │   ├── suppliers/
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── certifications/
│   │   │   │   │   │   └── route.js
│   │   │   │   │   ├── documents/
│   │   │   │   │   │   └── route.js
│   │   │   │   │   ├── feedback/
│   │   │   │   │   │   └── route.js
│   │   │   │   │   └── route.js
│   │   │   │   └── route.js
│   │   │   ├── transfers/
│   │   │   │   └── route.js            # CALL transfer_stock(...)
```

```
│   │   │   ├── users/
│   │   │   │   └── route.js
│   │   │   └── warehouses/
│   │   │       ├── [id]/
│   │   │       │   └── route.js
│   │   │       └── route.js
│   │   ├── dashboard/
│   │   │   └── page.js
│   │   ├── forbidden/
│   │   │   └── page.js
│   │   ├── inventory/
│   │   │   └── page.js
│   │   ├── login/
│   │   │   └── page.js
│   │   ├── products/
│   │   │   ├── [id]/
│   │   │   │   └── page.js
│   │   │   ├── new/
│   │   │   │   └── page.js
│   │   │   └── page.js
│   │   ├── purchase-orders/
│   │   │   ├── [id]/
│   │   │   │   └── page.js
│   │   │   ├── new/
│   │   │   │   └── page.js
│   │   │   └── page.js
│   │   ├── supplier-quality/
│   │   │   └── page.js
│   │   ├── suppliers/
│   │   │   └── [id]/
│   │   │       └── page.js
│   │   ├── layout.js                    # root layout + Navbar
│   │   └── page.js                      # home (redirects by role)
│   ├── components/                      # shared React components
│   │   ├── ErrorAlert.js
│   │   └── Navbar.js
│   ├── lib/                             # shared server code
│   │   ├── api.js                       # client fetch helper
│   │   ├── auth.js                      # getSession(), requireRole()
│   │   ├── db.js                        # PostgreSQL pool (pg), parameterised query()
│   │   ├── dberror.js                   # PostgreSQL error → HTTP 400
│   │   ├── mongodb.js                   # cached MongoClient
│   │   ├── mongoerror.js                # MongoDB error → HTTP 400/503
│   │   ├── pipelines.js                 # MongoDB aggregation pipelines
│   │   ├── session.js                   # JWT sign / verify (jose)
│   │   └── supplierDocs.js              # cross-DB helpers (upsert + link)
│   ├── scripts/                         # CLI tools
│   │   ├── clean-feedback.mjs           # cross-DB data clean-up
│   │   └── create-user.mjs              # create users with bcrypt hashes
│   ├── .env.example                     # config template (no real values)
│   ├── package.json
│   └── proxy.js                         # login + role check on every request
├── .gitignore                           # keeps .env files out of git
├── package.json
└── README.md                            # setup instructions
```

10. Few Sample Screenshots of Application Code and Demo

Real source files from the repository and the running application.

10.1 Application code

JS route.js ×

web › app › api › transfers › JS route.js

```
import { NextResponse } from "next/server";
import { query } from "@/lib/db";
import { requireRole } from "@/lib/auth";
import { dbErrorResponse } from "@/lib/dberror";

// POST /api/transfers  { product_id, from_warehouse, to_warehouse, quantity }
// The stored procedure does the real work in one transaction: it locks the rows,
// checks stock and capacity, and RAISEs an exception if anything is wrong.
export async function POST(request) {
  const { error } = await requireRole("ADMIN", "MANAGER");
  if (error) return error;

  const b = await request.json().catch(() => ({}));
  const toNum = (x) => (x === "" || x == null ? NaN : Number(x));
  const [product, from, to, qty] = [
    b.product_id,
    b.from_warehouse,
    b.to_warehouse,
    b.quantity,
  ].map(toNum);

  // Only checks that we got numbers; stock and capacity rules live in the database
  if (![product, from, to, qty].every(Number.isInteger)) {
    return NextResponse.json(
      { error: "Select a product, both warehouses and enter a whole-number quantity" },
      { status: 400 }
    );
  }

  try {
    await query("CALL transfer_stock($1::int, $2::int, $3::int, $4::int)", [
      product,
      from,
      to,
      qty,
    ]);
    return NextResponse.json({ ok: true });
  } catch (e) {
    return dbErrorResponse(e); // e.g. insufficient stock, capacity exceeded
  }
}
```

Figure 10.1: Stock transfer API: validates input, then calls the transfer_stock stored procedure; database errors (insufficient stock, capacity) are returned to the UI

JS route.js ×

web › app › api › purchase-orders › JS route.js

```
59    export async function POST(request) {
60      const { session, error } = await requireRole("ADMIN", "MANAGER");
61      if (error) return error;
62
63      const b = await request.json().catch(() => ({}));
64      const num = (x) => (x === "" || x == null ? null : Number(x));
65      const items = Array.isArray(b.items) ? b.items : [];
66
67      if (items.length === 0) {
68        return NextResponse.json({ error: "Add at least one item" }, { status: 400 });
69      }
70
71      const client = await pool.connect(); // a dedicated connection for the transaction
72      try {
73        await client.query("BEGIN");
74
75        const po = await client.query(
76          `INSERT INTO purchase_orders
77             (supplier_id, warehouse_id, created_by, status, order_date, expected_date)
78           VALUES ($1, $2, $3, 'ORDERED', CURRENT_DATE, $4)
79           RETURNING po_id`,
80          [num(b.supplier_id), num(b.warehouse_id), session.user_id, b.expected_date || null]
81        );
82        const poId = po.rows[0].po_id;
83
84        for (const it of items) {
85          await client.query(
86            `INSERT INTO po_items (po_id, product_id, quantity, unit_cost)
87             VALUES ($1, $2, $3, $4)`,
88            [poId, num(it.product_id), num(it.quantity), num(it.unit_cost)]
89          );
90        }
91
92        await client.query("COMMIT");
93        return NextResponse.json({ po_id: poId }, { status: 201 });
94      } catch (e) {
95        await client.query("ROLLBACK").catch(() => {});
96        return dbErrorResponse(e);
97      } finally {
98        client.release(); // always hand the connection back to the pool
99      }
100   }
```

Figure 10.2: Creating a purchase order: header and items inserted inside one transaction (BEGIN / COMMIT, ROLLBACK on error)

JS route.js ×

web › app › api › auth › login › JS route.js

```
import { NextResponse } from "next/server";
import bcrypt from "bcryptjs";
import { query } from "@/lib/db";
import { signSession, COOKIE_NAME } from "@/lib/session";

export async function POST(request) {
  const { username, password } = await request.json().catch(() => ({}));
  if (!username || !password) {
    return NextResponse.json({ error: "Username and password are required" }, { status: 400 });
  }

  const result = await query(
    "SELECT user_id, username, password_hash, role, supplier_id FROM users WHERE username = $1",
    [username]
  );
  const user = result.rows[0];

  let ok = false;
  if (user) {
    try {
      ok = await bcrypt.compare(password, user.password_hash);
    } catch {
      ok = false; // e.g. the placeholder hash is not a valid bcrypt hash
    }
  }
  if (!ok) {
    return NextResponse.json({ error: "Invalid username or password" }, { status: 401 });
  }

  const token = await signSession({
    user_id: user.user_id,
    username: user.username,
    role: user.role,
    supplier_id: user.supplier_id,
  });

  const res = NextResponse.json({ username: user.username, role: user.role });
  res.cookies.set(COOKIE_NAME, token, {
    httpOnly: true,                          // JavaScript cannot read it
    secure: process.env.NODE_ENV === "production", // HTTPS only when deployed
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 8,
  });
  return res;
}
```

Figure 10.3: Login: bcrypt password check and a signed JWT in an httpOnly cookie

JS proxy.js ×

web › JS proxy.js

```
import { NextResponse } from "next/server";
import { COOKIE_NAME, verifySession } from "@/lib/session";

// Pages that only certain roles may open
const PAGE_RULES = [
  { prefix: "/admin", roles: ["ADMIN"] },
  { prefix: "/inventory", roles: ["ADMIN", "MANAGER"] },
  { prefix: "/alerts", roles: ["ADMIN", "MANAGER"] },
  { prefix: "/supplier-quality", roles: ["ADMIN", "MANAGER"] },
  { prefix: "/products/new", roles: ["ADMIN", "MANAGER"] },
  { prefix: "/purchase-orders/new", roles: ["ADMIN", "MANAGER"] },
];

const PUBLIC_PATHS = ["/login", "/api/auth/login", "/api/health"];

export async function proxy(request) {
  const { pathname } = request.nextUrl;
  const token = request.cookies.get(COOKIE_NAME)?.value;
  const session = token ? await verifySession(token) : null;

  if (PUBLIC_PATHS.includes(pathname)) {
    // Already logged in: skip the login page
    if (pathname === "/login" && session) {
      return NextResponse.redirect(new URL("/", request.url));
    }
    return NextResponse.next();
  }

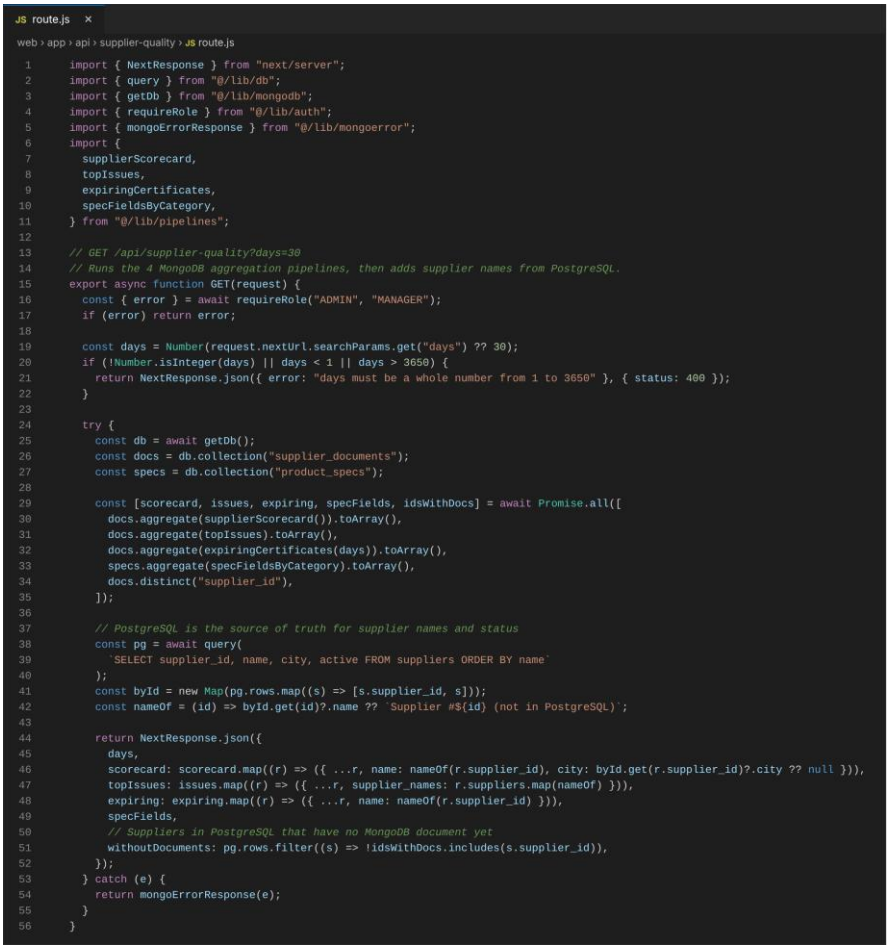
  if (!session) {
    if (pathname.startsWith("/api/")) {
      return NextResponse.json({ error: "Not logged in" }, { status: 401 });
    }
    return NextResponse.redirect(new URL("/login", request.url));
  }

  const rule = PAGE_RULES.find((r) => pathname.startsWith(r.prefix));
  if (rule && !rule.roles.includes(session.role)) {
    return NextResponse.redirect(new URL("/forbidden", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/((?!_next|favicon.ico|.*\\..*).*)"],
};
```

Figure 10.4: proxy.js: runs before every request; redirects to login and blocks pages by role



```
JS route.js ×
web › app › api › supplier-quality › JS route.js
1   import { NextResponse } from "next/server";
2   import { query } from "@/lib/db";
3   import { getDb } from "@/lib/mongodb";
4   import { requireRole } from "@/lib/auth";
5   import { mongoErrorResponse } from "@/lib/mongoerror";
6   import {
7     supplierScorecard,
8     topIssues,
9     expiringCertificates,
10    specFieldsByCategory,
11  } from "@/lib/pipelines";
12
13  // GET /api/supplier-quality?days=30
14  // Runs the 4 MongoDB aggregation pipelines, then adds supplier names from PostgreSQL.
15  export async function GET(request) {
16    const { error } = await requireRole("ADMIN", "MANAGER");
17    if (error) return error;
18
19    const days = Number(request.nextUrl.searchParams.get("days") ?? 30);
20    if (!Number.isInteger(days) || days < 1 || days > 3650) {
21      return NextResponse.json({ error: "days must be a whole number from 1 to 3650" }, { status: 400 });
22    }
23
24    try {
25      const db = await getDb();
26      const docs = db.collection("supplier_documents");
27      const specs = db.collection("product_specs");
28
29      const [scorecard, issues, expiring, specFields, idsWithDocs] = await Promise.all([
30        docs.aggregate(supplierScorecard()).toArray(),
31        docs.aggregate(topIssues).toArray(),
32        docs.aggregate(expiringCertificates(days)).toArray(),
33        specs.aggregate(specFieldsByCategory).toArray(),
34        docs.distinct("supplier_id"),
35      ]);
36
37      // PostgreSQL is the source of truth for supplier names and status
38      const pg = await query(
39        `SELECT supplier_id, name, city, active FROM suppliers ORDER BY name`
40      );
41      const byId = new Map(pg.rows.map((s) => [s.supplier_id, s]));
42      const nameOf = (id) => byId.get(id)?.name ?? `Supplier #${id} (not in PostgreSQL)`;
43
44      return NextResponse.json({
45        days,
46        scorecard: scorecard.map((r) => ({ ...r, name: nameOf(r.supplier_id), city: byId.get(r.supplier_id)?.city ?? null })),
47        topIssues: issues.map((r) => ({ ...r, supplier_names: r.suppliers.map(nameOf) })),
48        expiring: expiring.map((r) => ({ ...r, name: nameOf(r.supplier_id) })),
49        specFields,
50        // Suppliers in PostgreSQL that have no MongoDB document yet
51        withoutDocuments: pg.rows.filter((s) => !idsWithDocs.includes(s.supplier_id)),
52      });
53    } catch (e) {
54      return mongoErrorResponse(e);
55    }
56  }
```

Figure 10.5: Supplier-quality API: four MongoDB aggregation pipelines combined with supplier names from PostgreSQL

```
JS db.js ×
web › lib › JS db.js
1   import { Pool } from "pg";
2
3   // Reuse one pool across hot reloads in dev
4   const pool =
5     globalThis.pgPool ?? new Pool({ connectionString: process.env.DATABASE_URL });
6   if (process.env.NODE_ENV !== "production") globalThis.pgPool = pool;
7
8   export { pool };
9
10  // Parameterised queries only: query("SELECT * FROM products WHERE product_id = $1", [id])
11  export function query(text, params) {
12    return pool.query(text, params);
13  }
```

Figure 10.6: lib/db.js: one shared PostgreSQL connection pool; the connection string comes from an environment variable

10.2 Application demo

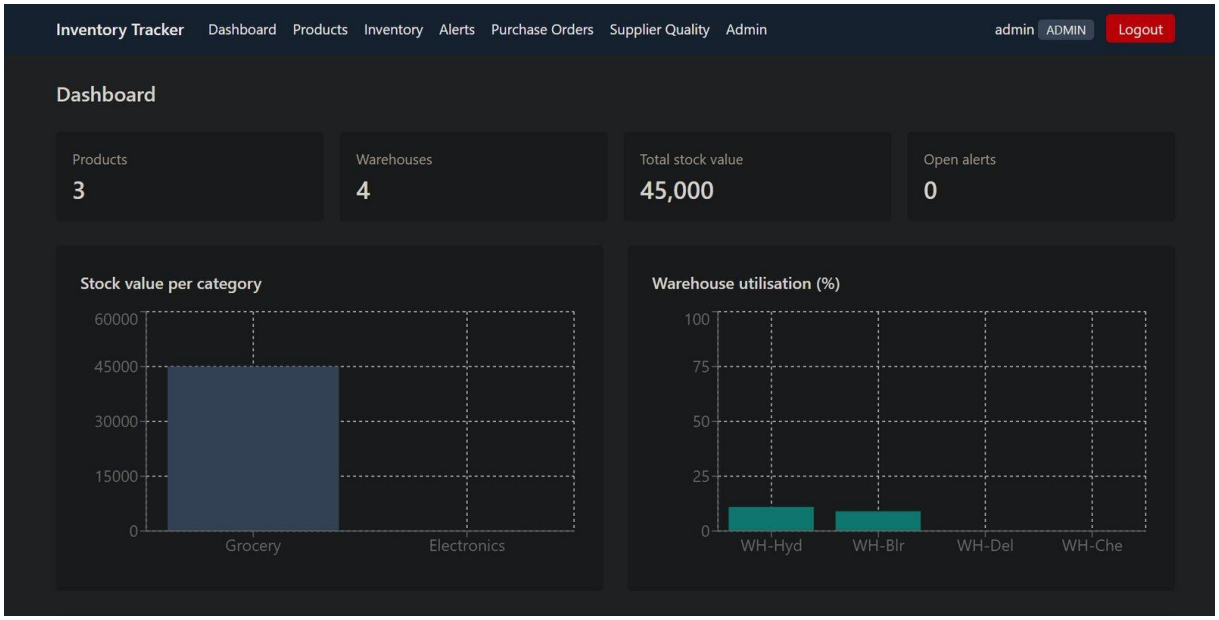


Figure 10.7: Dashboard: totals (scalar subqueries over a view), stock value per category (GROUP BY) and warehouse utilisation (LEFT JOIN with a computed ratio)

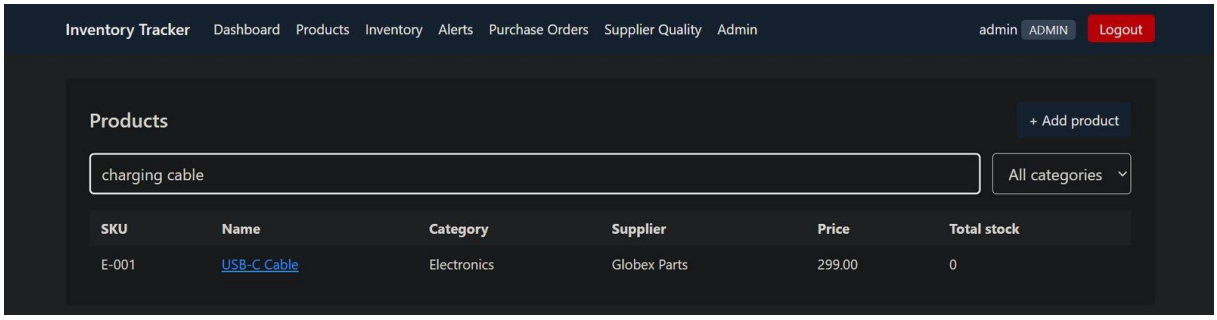


Figure 10.8: Full-text search: "charging cable" finds the USB-C cable through the GIN-indexed tsvector column

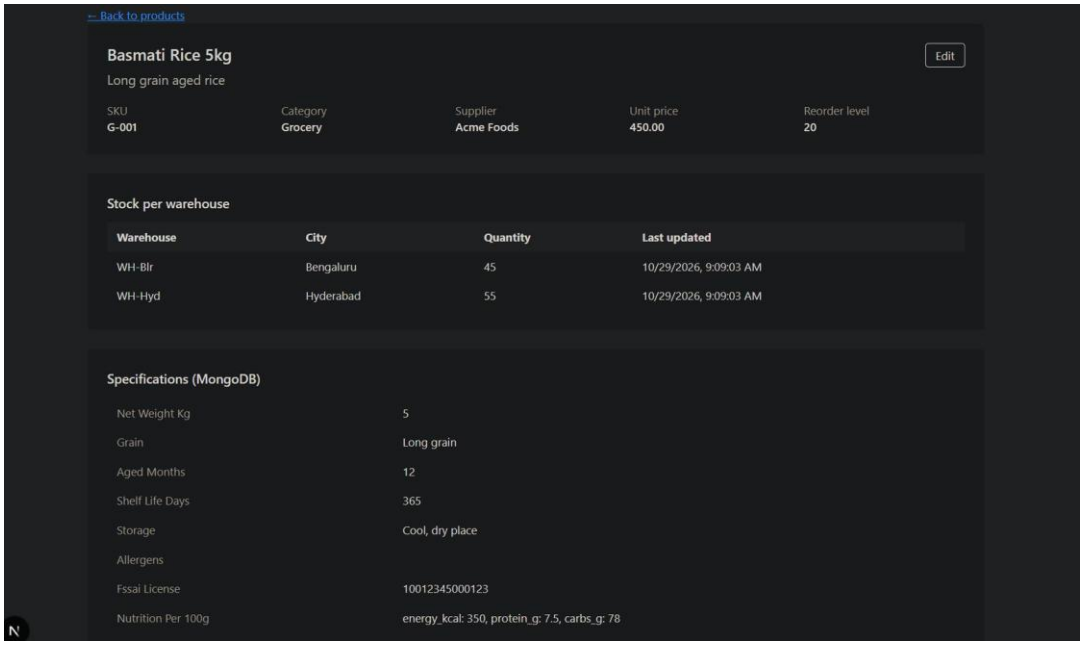


Figure 10.9: Product detail: PostgreSQL data (SKU, price, stock per warehouse) combined with category-specific specifications from MongoDB

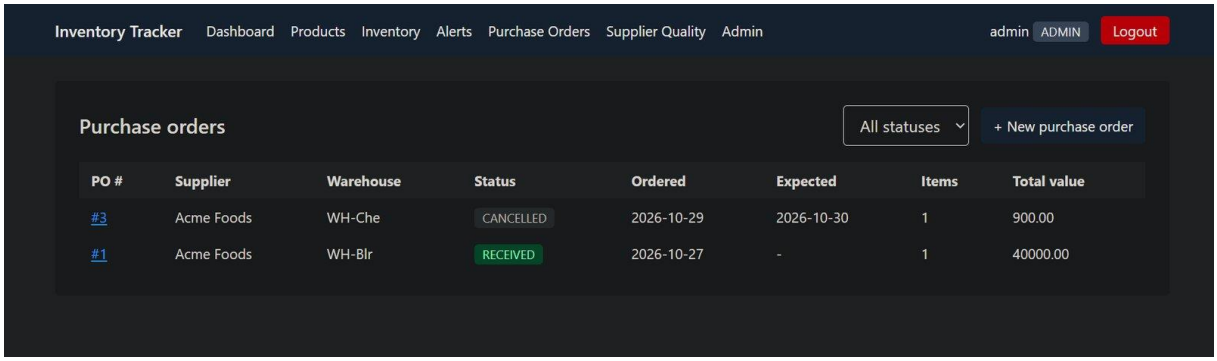


Figure 10.10: Purchase orders with totals (JOIN + GROUP BY + SUM). PO #1 was received through the receive_purchase_order procedure; PO #3 was cancelled.

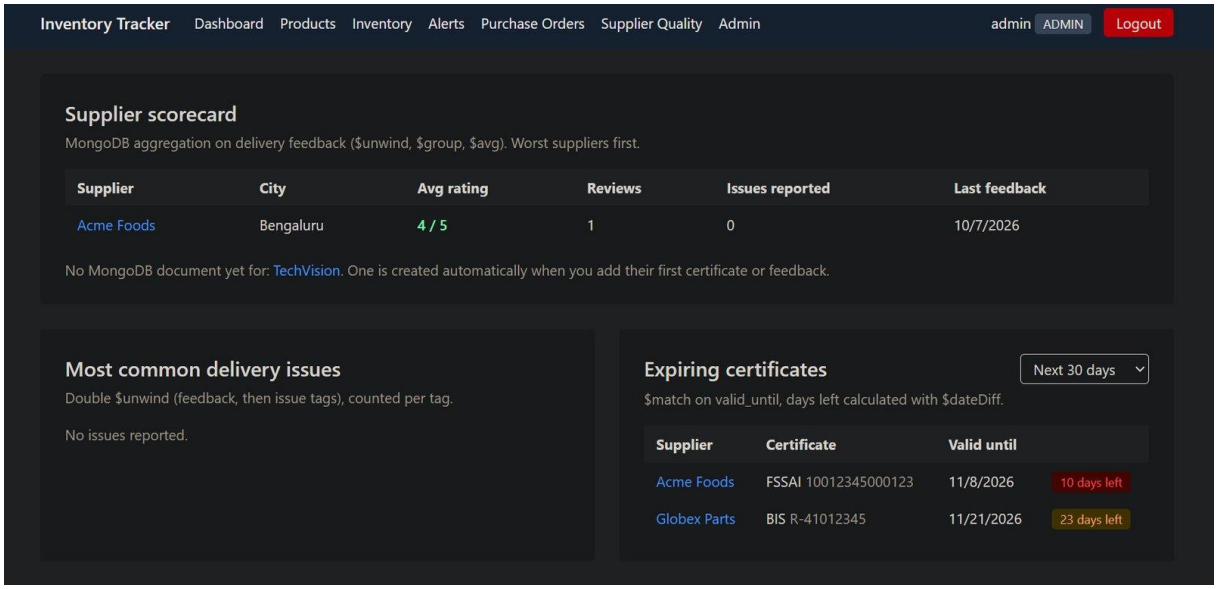


Figure 10.11: Supplier Quality: scorecard and expiring certificates from MongoDB aggregation pipelines; suppliers without a MongoDB document are listed

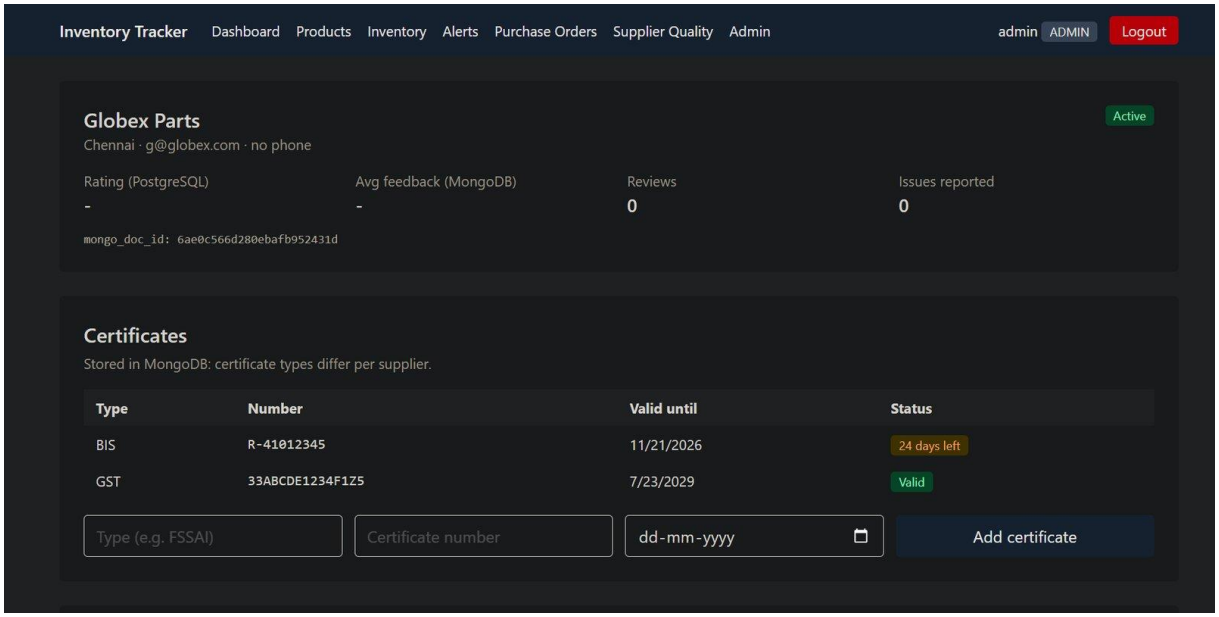


Figure 10.12: Supplier page: PostgreSQL supplier record and linked mongo_doc_id, with certificates stored in MongoDB

11. Evaluation Rubrics: Completion Status

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	**Completed**
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	**Completed**
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	**Completed**
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	**Completed**
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	**Completed**
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	**Completed**
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	**Completed**

11.1 Detailed completion checklist

Each requirement of the project brief, its status and where it is shown in this report.

#	Rubric item (project brief)	Status	Evidence
1	Project title: unique, real-world, not simple CRUD	**Completed**	Sections 1, 1.2
2	Team of 2 with clearly defined responsibilities (database + application)	**Completed**	Sections 1.3, 11.2; GitHub commits
3	Database requirement: SQL (PostgreSQL) + NoSQL	**Completed**	PostgreSQL 18 + MongoDB Atlas (Section 2)
4	Database design: ER diagram → relational schema; PK, FK, relationships, constraints; NoSQL data model, indexes, retrieval	**Completed**	Sections 3, 4, 7.1–7.6
5	SQL implementation: CREATE, ALTER, DROP; PK/FK, constraints, indexes, views	**Completed**	Sections 5.1, 5.2, 5.3

#	Rubric item (project brief)	Status	Evidence
6	DML: INSERT, UPDATE, DELETE, SELECT; JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	**Completed***	Sections 6.1–6.8. Business records use soft delete (deactivate / cancel) to keep history; DELETE with ON DELETE CASCADE shown at SQL level
7	≥ 4 advanced SQL features + ≥ 1 NoSQL feature	**Completed**	6 SQL (transactions, procedures, triggers, views, indexing, full-text search) + 2 NoSQL (aggregation, schema validation): Section 6.11
8	Vector database / semantic search	Not used (optional)	Optional; Option 1 (SQL + NoSQL) chosen. Listed as a future enhancement (Section 11.4)
9	Web interface in Next.js with forms, tables, search, filters, dashboards	**Completed**	Sections 9, 10.2
10	Database–application integration through an API layer; no hard-coded credentials; environment variables	**Completed**	Sections 2.2, 9, 10.1 (Fig. 10.6)
11	Authentication and role-based access (≥ 2 roles)	**Completed**	3 roles: Admin, Manager, Supplier (Fig. 10.3, 10.4, 8.8)
12	Input validation at database level; data consistency and integrity	**Completed**	Constraints, triggers, $jsonSchema; Sections 5, 7.2, 8.2
13	Security: passwords and credentials not stored in plain text	**Completed**	bcrypt hashes; .env.local git-ignored; parameterised SQL (Section 2.4)
14	Project documentation (soft copy)	**Completed**	This report
15	Git/GitHub: source code, individual commits, README	**Completed**	github.com/RahulPrasanna22/DBMS-Mini_Project
16	Demonstration: design, implementation, queries, interface, integration, advanced features, test cases, contributions	**Completed**	Sections 5–8, 10, 11.2
17	Originality: original design and implementation	**Completed**	Section 1.2 (Novelty)
18	Level 3: beyond CRUD; database choice justified by the data	**Completed**	Sections 1.2, 2.3

Summary: 17 of 18 items completed; item 8 is optional and was not used. *See the note in the Evidence column.

11.2 Individual contributions

Member	Database	Application and other
Prithvish Adhikari	• PostgreSQL schema: tables, constraints, indexes • Triggers, stored procedures, views • Seed and test script • MongoDB Atlas setup, collections, `$jsonSchema` validators, indexes • Aggregation pipelines • PostgreSQL–MongoDB link script	• Supplier Quality dashboard and supplier documents pages • Feedback and certificate APIs (cross-database checks, upsert, rating sync) • MongoDB error mapping, feedback clean-up script • Production-build fix, project README • Credential clean-up and Atlas password change

Member	Database	Application and other
Rahul Prasanna	• PostgreSQL integration: connection pool, parameterised queries • Analytical SQL for the dashboard (GROUP BY, HAVING, subqueries) • Transactional purchase-order creation • Database testing and the SQL test run	• Next.js project setup and structure • Authentication (bcrypt, JWT cookie), roles, proxy • Products, search, inventory and transfers, alerts • Purchase orders, dashboard with charts, admin pages • Application testing and screenshots

Individual commits are visible in the repository history. Problem analysis, the ER design and the choice of databases were done together.

11.3 Deliverables

- **Completed DBMS Project Report:** this document.
- **GitHub repository (project source code):** https://github.com/RahulPrasanna22/DBMS-Mini_Project. Setup steps are in `README.md`.

11.4 Future enhancements

1. **Semantic search over feedback** using pgvector or MongoDB Atlas Vector Search, for example "find suppliers with packaging problems" even when the wording differs.
2. **Demand forecasting:** use stock-movement history to predict reorder dates and quantities instead of a fixed reorder level.
3. **Sales orders and invoicing** with GST, so stock goes out as well as in.
4. **Notifications:** email or WhatsApp messages for low-stock alerts and certificates about to expire.
5. **Barcode scanning** on mobile for receiving goods and stock counts.
6. **Batch and expiry tracking** for perishable goods, with first-expiry-first-out picking.
7. **Supplier self-service:** suppliers upload certificates and confirm delivery dates from their portal.
8. **Audit log** of who changed what, and a **cloud deployment** with HTTPS and managed PostgreSQL.

Annexure A: Stock Report

Stock position, warehouse utilisation and stock movements from the live application (30 October 2026).

PRODUCTS	WAREHOUSES	TOTAL STOCK VALUE	OPEN ALERTS
3	**3**	**₹47,250**	**1**
G-001, G-003, E-001	Hyderabad, Bengaluru, Chennai	105 units in stock	G-001 low at WH-Blr

This report presents the current stock position of the Smart Inventory & Supply Chain Tracker. The figures are taken from the live application (admin dashboard), which reads them from PostgreSQL using the `v_stock_summary` view, aggregate queries over inventory, and the `stock_movements` audit log. Stock changes only through the `transfer_stock` and `receive_purchase_order` procedures, which update inventory and the movement log in the same transaction.

A.1 Stock overview

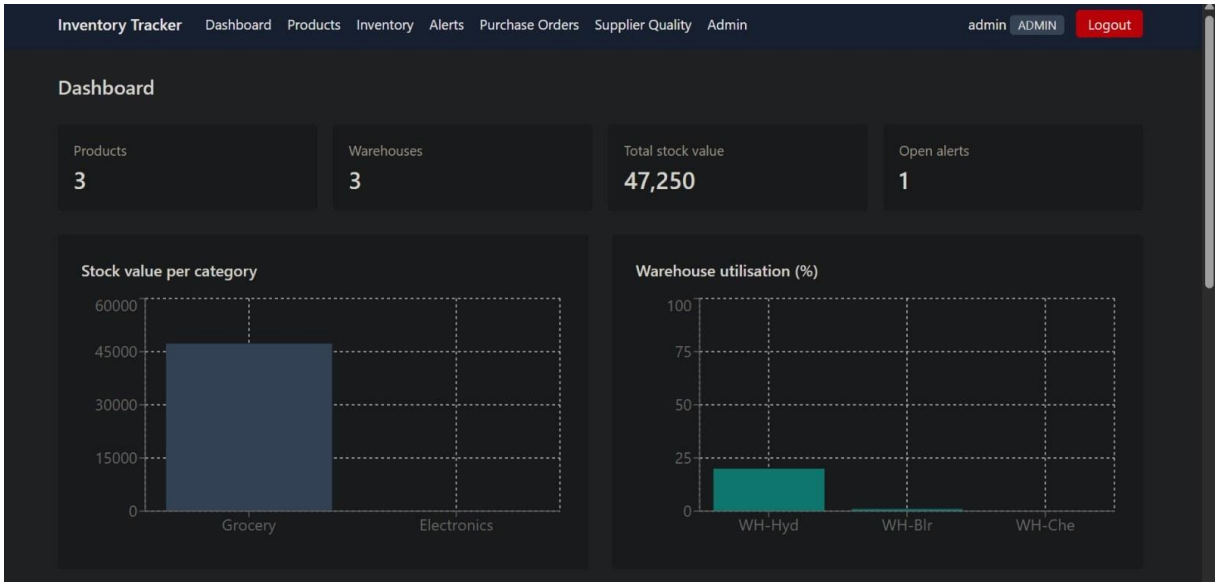


Figure A.1: Admin dashboard: totals, stock value per category, and warehouse utilisation chart

- **Total stock value is ₹47,250.** It is computed in the `v_stock_summary` view as quantity × unit price: 105 units of Basmati Rice 5kg (G-001) × ₹450.
- **All of the value is in Grocery.** Electronics holds no stock: the USB-C Cable (E-001) has not been ordered yet.
- **One low-stock alert is open.** It was created automatically by the `trg_low_stock` trigger (see Section A.3).

A.2 Warehouse utilisation



Warehouse utilisation

SUM(quantity) / capacity per warehouse.

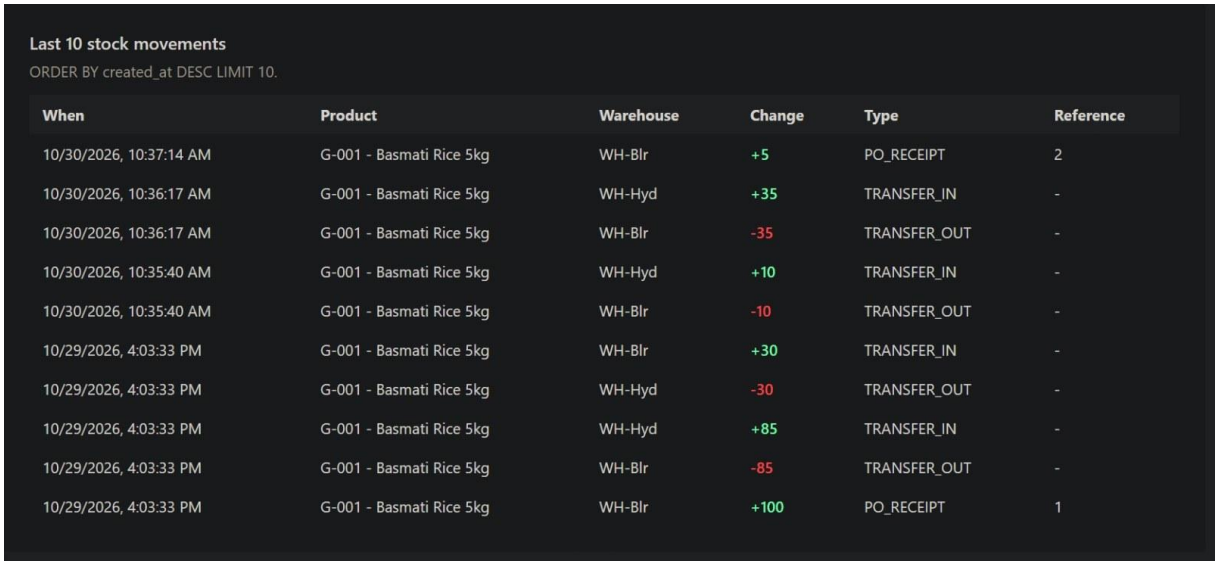
Warehouse	Units stored	Capacity	Utilisation
WH-Hyd	100	500	20.0%
WH-Blr	5	500	1.0%
WH-Che	0	40	0.0%

Figure A.2: Units stored against capacity per warehouse (SUM(quantity) / capacity, using LEFT JOIN so empty warehouses still appear)

Warehouse	City	Units stored	Capacity	Free space	Utilisation	Remark
WH-Hyd	Hyderabad	100	500	400	20.0%	Holds most of the stock (95%)
WH-Blr	Bengaluru	5	500	495	1.0%	**Below reorder level**
WH-Che	Chennai	0	40	40	0.0%	Empty; small site (capacity 40)
Total		**105**	**1,040**	**935**	**10.1%**	

Capacity is enforced by the `trg_capacity` trigger. No warehouse can be filled beyond its capacity, whether by a transfer or by receiving a purchase order.

A.3 Stock movements



Last 10 stock movements

ORDER BY created_at DESC LIMIT 10.

When	Product	Warehouse	Change	Type	Reference
10/30/2026, 10:37:14 AM	G-001 - Basmati Rice 5kg	WH-Blr	+5	PO_RECEIPT	2
10/30/2026, 10:36:17 AM	G-001 - Basmati Rice 5kg	WH-Hyd	+35	TRANSFER_IN	-
10/30/2026, 10:36:17 AM	G-001 - Basmati Rice 5kg	WH-Blr	-35	TRANSFER_OUT	-
10/30/2026, 10:35:40 AM	G-001 - Basmati Rice 5kg	WH-Hyd	+10	TRANSFER_IN	-
10/30/2026, 10:35:40 AM	G-001 - Basmati Rice 5kg	WH-Blr	-10	TRANSFER_OUT	-
10/29/2026, 4:03:33 PM	G-001 - Basmati Rice 5kg	WH-Blr	+30	TRANSFER_IN	-
10/29/2026, 4:03:33 PM	G-001 - Basmati Rice 5kg	WH-Hyd	-30	TRANSFER_OUT	-
10/29/2026, 4:03:33 PM	G-001 - Basmati Rice 5kg	WH-Hyd	+85	TRANSFER_IN	-
10/29/2026, 4:03:33 PM	G-001 - Basmati Rice 5kg	WH-Blr	-85	TRANSFER_OUT	-
10/29/2026, 4:03:33 PM	G-001 - Basmati Rice 5kg	WH-Blr	+100	PO_RECEIPT	1

Figure A.3: Last 10 stock movements from the stock_movements audit log (ORDER BY created_at DESC LIMIT 10)

Every change to stock is logged. Replaying the ten movements in time order gives the running balance of G-001 in each warehouse:

Date / time	Movement	Caused by	WH-Blr	WH-Hyd	Alert (reorder level 20)
29 Oct, 4:03 PM	Receipt +100 at WH-Blr	PO #1 received	100	0	
29 Oct, 4:03 PM	Transfer 85: Blr → Hyd	transfer_stock	15	85	**raised:** Blr fell to 15
29 Oct, 4:03 PM	Transfer 30: Hyd → Blr	transfer_stock	45	55	**resolved:** Blr back to 45
30 Oct, 10:35 AM	Transfer 10: Blr → Hyd	transfer_stock	35	65	
30 Oct, 10:36 AM	Transfer 35: Blr → Hyd	transfer_stock	0	100	**raised:** Blr fell to 0
30 Oct, 10:37 AM	Receipt +5 at WH-Blr	PO #2 received	5	100	still open (5 ≤ 20)
Closing balance			**5**	**100**	**1 open alert**

Reconciliation: the closing balances rebuilt from the movement log (WH-Blr 5, WH-Hyd 100) **match the utilisation table exactly**, and 105 units × ₹450 = ₹47,250 matches the dashboard. Receipts total 105 units (POs #1 and #2). Transfers only move stock between warehouses, so they do not change the total.

A.4 Findings and recommended actions

Findings

- Stock is concentrated in Hyderabad: 100 of 105 units (95%).
- Bengaluru is down to 5 units of G-001, below its reorder level of 20. This is the one open alert.
- Overall utilisation is low (10.1% of 1,040 units of capacity), so there is plenty of space.
- Electronics has no stock, and the USB-C Cable has never been ordered.

Recommended actions

Restock Bengaluru: either transfer about 35 units from Hyderabad (which has spare stock) or raise a purchase order. The reorder view suggests 2 × 20 − 5 = 35 units, available as "Create PO" on the Alerts page.

Balance stock between Hyderabad and Bengaluru to reduce the risk of running out in one city.

Place a first order for the USB-C Cable if it is meant to be sold.